IMPERIAL

Adaptive Acquisition: A Reinforcement Learning Framework for Autonomous MRI Sequence Optimization

Author

A. Allilaire

CID: 02372975

Supervised by

Dr Andreas Wetscherek

Prof. Wayne Luk

Second Marker

Prof. Ben Glocker

An Interim Report submitted in fulfillment of requirements for the degree of
Bachelor of Engineering in Computing

Department of Computing
Imperial College London

2026

Contents

1 Introduction

Contents

1.1 Introduction

In 2021, there were 395,000 cases of cancer in the UK, with 1 in 2 people in the UK being diagnosed with cancer in their lifetime. [1] Radiotherapy (RT) is a crucial component of cancer treatment; indeed, it is required for more than 50% of cancer patients. [2]

Image-guided radiation therapy (IGRT) is used to increase the precision of treatment and ensure accurate dose delivery. Conventionally, images are taken using a high-resolution X-ray technique called Cone Beam Computed Tomography. This is still the most common technique. [3]

However, poor image quality (caused by excessive scattering of X-ray photons) and ionising radiation are key drawbacks. Instead, Magnetic Resonance Imaging (MRI) can be used to improve soft tissue contrast (vital for liver or pancreatic cancer) and increase the accuracy of tumor target delineation. MR-guided radiation therapy (MRgRT) has been shown to significantly improve patient prognosis. [3]

MRI and a linear accelerator (the source of radiation) have been coupled into a hybrid Magnetic Resonance Accelerator (MR-Linac), the latest innovation in IGRT. The MR-Linac allows on-table adaptation of treatment plans, to respond to tumour movements and progression. However, one of the clinical bottlenecks is the extra time required for image acquisition. [4] The high-resolution images needed for accurate

delineation require extended scan time. This also increases the risk of motion artifacts (inaccuracies due to patient movement).

There has been significant research into accelerating image acquisition, focusing mainly on using Deep Learning (computational neural network approaches) for image reconstruction (translating the raw MR signal into an image). Leading to significant improvements to image robustness, artifact reduction and image quality when using undersampled data.

There is now a growing interest in using DL and Reinforcement Learning (agent-based approaches) to develop novel pulse sequences, which are currently designed by MR experts. Novel pulse sequences have shown promise in speeding up image acquisition while maintaining image quality. [5]

1.2 Research challenges and achievements

This project studies whether reinforcement learning can be used to design adaptive quantitative MRI (qMRI) sequences. Rather than only reconstructing an image from a fixed acquisition, the aim is to let an agent choose the next acquisition based on what has already been measured. This makes the problem attractive, but also exposes three practical challenges.

C1: simulator validation for qMRI. The RL agent learns entirely from simulated MRI data, so the simulator must be numerically trustworthy before any learned policy can be evaluated. A visually plausible image is not sufficient: for qMRI, the pipeline must recover the correct quantitative tissue parameters, such as T_1 and T_2, from a known phantom.

C2: computational efficiency under expensive simulation. A live Bloch-equation simulator is much slower than the analytical environments normally used in reinforcement learning. This makes training compute-bound. The project therefore needs a multi-fidelity strategy: fast, lower-fidelity configurations for learning, with more accurate simulations reserved for validation.

C3: adaptive sequence design from tissue-parameter estimates. The final objective is not simply to run a fixed protocol faster, but to choose acquisitions based on the current estimate of the tissue parameters. The agent should use the signals observed so far to decide which inversion times, echo times, or flip angles are most informative next.

The report is organised around three corresponding achievements.

A1: a validated digital twin and simulator pipeline. The project built `MRISystemPhantom.jl`, a configurable digital twin of the QalibreMD MRI system phantom, and used it to validate the KomaMRI simulation, reconstruction, and fitting pipeline. This validation found two floating-point bugs

in KomaMRI's long-sequence timing code, which were reduced to minimal reproducers and submitted upstream.

A2: a compute-aware, multi-fidelity RL environment. The project developed a Python/Julia Gymnasium environment that connects reinforcement learning agents to KomaMRI while managing the cost of simulation. The key design is to separate fast training configurations from higher-fidelity evaluation, so that policies can be trained within the available compute budget and then checked under more realistic simulation settings.

A3: adaptive qMRI sequence design. The project implements adaptive sequence-design experiments in which the agent selects acquisition parameters using the current tissue-parameter estimates. This links the validated simulator and multi-fidelity training loop to the central research question: whether learned adaptive protocols can outperform fixed conventional qMRI sequences.

2

Background & Related Work

Contents

It is necessary to understand the current process of MRI Pulse Sequence design before attempting to use DL or RL to improve upon it. This requires an understanding of MRI-Physics; a short introduction follows.

2.1 Principles of Magnetic Resonance Imaging

The signal generated by an MRI uses the Nuclear Magnetic Resonance (NMR) of protons. All subatomic particles have intrinsic angular momentum, which is a quantum property called spin. For example, a

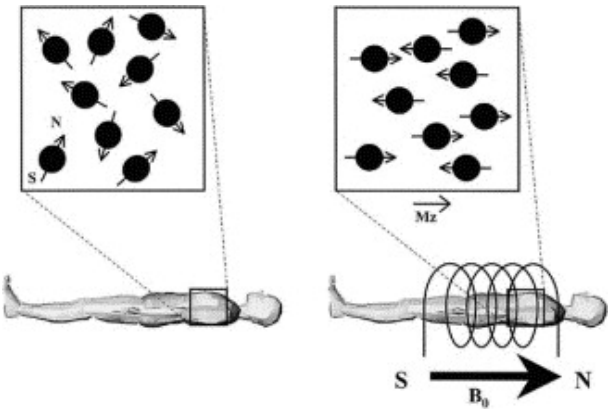


Figure 2.1: Protons align in parallel or antiparallel to the external field. Parallel alignment is a lower energy state, thus slightly more popular, leading to a net magnetisation vector (Mz). Figure from [7]

proton has a spin-$\frac{1}{2}$, while a neutron has a spin$\frac{1}{2}$. For a nucleus to have a net spin, it cannot have the same number of neutrons and protons.

A Hydrogen nucleus is made of a single proton, so it has a net spin-$\frac{1}{2}$. The angular momentum, combined with the mass and charge of a proton, results in a magnetic moment μ. When a uniform magnetic field (B_0) is applied, protons will align their magnetic axis with the field. (See Figure 2.1) However, a proton wobbles around the field's axis, which is called precession. Essentially, the proton behaves like a microscopic bar magnet rotating around its axis.[6]

The frequency of this rotation is known as the Larmor frequency (ω_0) and is directly proportional to the magnetic field (B_0). The constant of proportionality is called the gyromagnetic ratio (γ). For protons, $\gamma/2\pi = 42.58$ MHz T^{-1}. [7]

$$\omega_0 = \gamma B_0 \tag{2.1}$$

Here γ is the angular-frequency value. In the KomaMRI simulations later in this thesis, the cycle-frequency convention is used instead, with $\gamma = 42.58$ MHz T^{-1}. Accidentally using the angular-frequency value in this context introduces an extra factor of 2π, which shrinks the encoded field of view by 2π.

2.1.1 RF Pulses

The overall net magnetisation from all protons is represented by the vector $\mathbf{M} = (M_x, M_y, M_z)$. At equilibrium, M_z is aligned with the static B_0 field. The transverse components M_x and M_y, which rotate about B_0, are usually grouped into the complex notation $M_{xy} = M_x + iM_y$.

The receiver coils measure transverse magnetisation M_{xy}, not longitudinal magnetisation M_z directly. A perpendicular radio-frequency field B_1, applied at the Larmor frequency, tips some or all of M_z into the transverse plane. This creates M_{xy}, which precesses about B_0 and induces a voltage in receiver coils

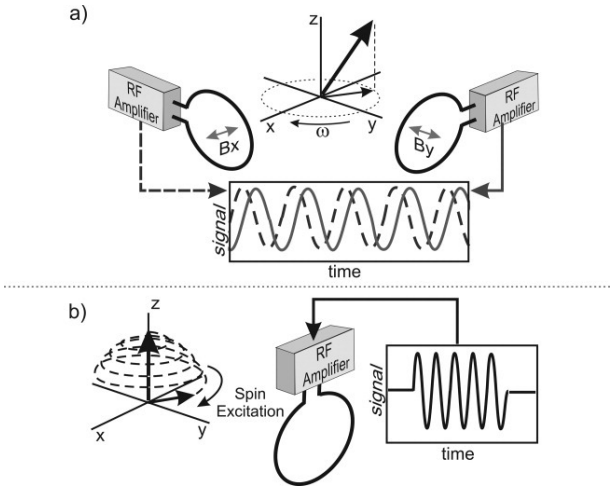


Figure 2.2: A voltage is induced via electromagnetic induction, making an MR signal. (A) The coils are perpendicular and can measure the phase and amplitude of the transverse magnetic vector. (B) RF pulses progressively "flip" the z-axis magnetisation vector into the transverse plane. Figure from [6]

placed perpendicular to the main field, as shown in Figure 2.2. The B_1 pulses are called radio-frequency (RF) pulses because the Larmor frequency lies in the radio-frequency range. [7][6]

The resulting angle between M and B_0 is called the flip angle (α) and can be calculated using:

$$\alpha = \gamma \int B_1(t)\, dt \tag{2.2}$$

2.1.2 T1, T2 & T2* Relaxation

After an RF pulse is switched off, M_z recovers towards thermal equilibrium along B_0, while M_{xy} decays in the transverse plane (the free induction decay). These are independent processes: longitudinal relaxation transfers energy back to the surrounding lattice, while transverse relaxation describes loss of phase coherence between spins.

Longitudinal, or spin-lattice, relaxation is characterised by T_1: the time taken for M_z to recover 63% of the way back to its equilibrium value M_0 after excitation. Because this recovery depends on energy exchange with the local molecular environment, T_1 is an intrinsic tissue parameter.[7]

Transverse, or spin-spin, relaxation describes the decay of M_{xy} as spins lose phase coherence after excitation. Irreversible dephasing from spin-spin interactions is characterised by T_2, while reversible dephasing from static B_0 inhomogeneity is written as T_2'. The observed transverse decay T_2^* combines both effects:

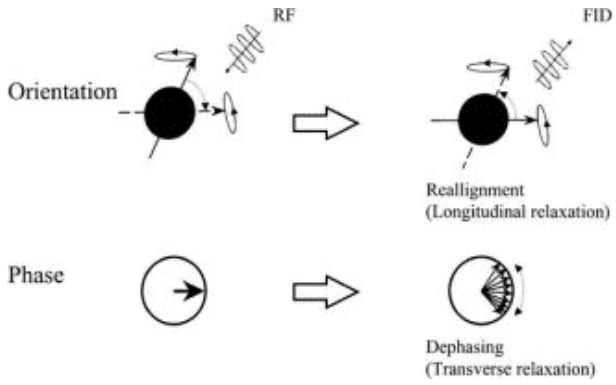


Figure 2.3: **Upper Row:** Longitudinal relaxation (decreasing the flip angle).
Lower Row: Transverse relaxation (spin dephasing) Figure from [7]

$$\frac{1}{T_2^*} = \frac{1}{T_2} + \frac{1}{T_2'}. \tag{2.3}$$

A 180° refocusing pulse reverses the static dephasing term T_2', which is why spin-echo sequences measure a signal governed primarily by T_2 relaxation. In biological tissue, T_2 is generally shorter than T_1.[6]

Bloch-equation model. The Bloch equations formalise the relaxation behaviour above. In the rotating frame at ω_0, with equilibrium magnetisation M_0 along $+z$, the relaxation dynamics can be written as:

$$\frac{dM_z}{dt} = -\frac{M_z - M_0}{T_1}, \qquad \frac{dM_{xy}}{dt} = -\frac{M_{xy}}{T_2} + i\Delta\omega M_{xy} \tag{2.4}$$

Here $\Delta\omega$ is the local off-resonance frequency relative to the rotating frame. The first equation describes longitudinal recovery: M_z exponentially returns to M_0 with time constant T_1. The second describes transverse decay: the magnitude of M_{xy} falls with time constant T_2, while off-resonance produces additional phase rotation.

Between RF pulses, these independent differential equations have the free-precession solutions:

$$M_z(t) = M_0 + (M_z(0) - M_0)\, e^{-t/T_1}, \qquad |M_{xy}(t)| = |M_{xy}(0)|\, e^{-t/T_2} \tag{2.5}$$

These are the signal-decay relationships used by the conventional quantitative MRI fitting methods used later in this work. Under the hard-pulse approximation, an RF pulse is treated as an instantaneous rotation by flip angle α about a transverse axis. If the longitudinal magnetisation immediately before excitation is M_z^-, the pulse leaves $\cos(\alpha)M_z^-$ on the longitudinal axis and tips $\sin(\alpha)M_z^-$ into the transverse

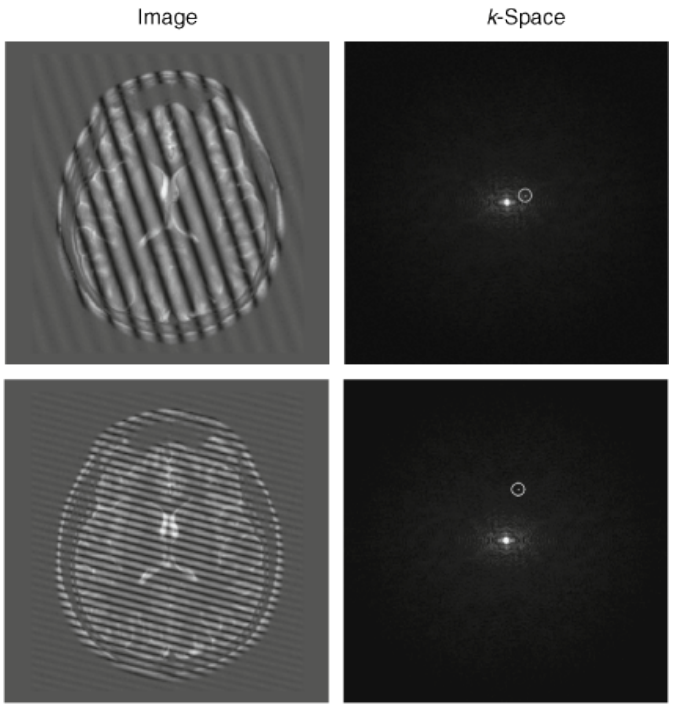


Figure 2.4: Image space and k-space are Fourier-pair representations of the same data: an MR image can be viewed either as a matrix of intensities $I(x, y)$ or as spatial-frequency amplitudes $S(k_x, k_y)$. Each k-space point weights a sinusoidal wave across the field of view; the vector angle (k_x, k_y) sets the wave-pattern orientation, and its distance from the origin sets the spatial frequency. The centre of k-space contains low-spatial-frequency contrast and mean signal, while the outer samples contain high-spatial-frequency detail. Figure from [8].

plane. The received signal is proportional to this transverse component.

After readout, the sequence models used in this work assume that residual transverse magnetisation is removed by gradient spoiling or by relaxation before the next acquisition block. The only state carried between shots is therefore the scalar longitudinal magnetisation, updated by RF rotations and T_1 recovery. This reduces the Bloch dynamics to simple recurrence relations; without this assumption, transverse coherences would need to be tracked explicitly using a coherence-pathway method such as Extended Phase Graphs (EPG).

2.1.3 Spatial Encoding & K-Space

Relaxation determines how much signal a spin can produce, but an image also requires spatial localisation. MRI does not measure image pixels directly. Instead, the scanner samples k-space: a spatial-frequency representation of the image, shown in Figure 2.4. The centre sample of k-space, $S(0, 0)$, has no gradient moment along either axis, so all spins contribute without a spatial phase ramp. It is therefore the integral of the whole image, or mean signal, conventionally called the DC term.

Let G_x, G_y, and G_z denote magnetic-field gradients along the scanner axes. These gradients make the

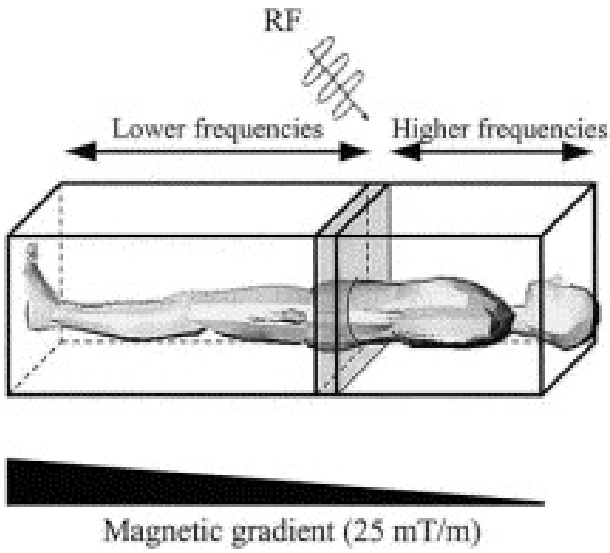


Figure 2.5: Slice selection using a narrow-band RF pulse applied during a gradient. Only spins whose shifted Larmor frequencies lie within the RF bandwidth are excited. Figure from [7].

Larmor frequency position-dependent by varying the local field strength, providing the spatial information missing from the raw MR signal.

The G_z gradient performs slice selection. During the RF pulse, G_z maps z-position to Larmor frequency, so a narrow-band RF pulse excites only the spins whose frequencies lie within the RF bandwidth. This selects a thin slice of the object, as shown in Figure 2.5.

The in-plane coordinates are then encoded using G_y and G_x. A short G_y pulse before readout provides phase encoding: it gives spins at different y-positions different accumulated phases. A G_x gradient is kept on during ADC readout for frequency encoding: successive samples trace spatial frequency along k_x. Repeating the sequence with different G_y amplitudes steps through k_y, so each repetition fills a different row of k-space.

For a gradient waveform $\mathbf{G}(t)$, the sampled k-space location is determined by the accumulated gradient moment:

$$\mathbf{k}(t) = \gamma \int_0^t \mathbf{G}(\tau)\, d\tau. \tag{2.6}$$

Stronger or longer gradients move the acquisition further through k-space. At a given k-space location, the acquired signal is

$$S(\mathbf{k}) = \int m(\mathbf{r}) e^{-i2\pi \mathbf{k}\cdot\mathbf{r}}\, d\mathbf{r}. \tag{2.7}$$

Here $m(\mathbf{r})$ is the transverse magnetisation at position $\mathbf{r}$. The exponential term applies a position-dependent phase rotation to each complex M_{xy} contribution before the receiver sums over the object. Changing $\mathbf{k}$ changes this phase pattern, so each measurement extracts a different spatial-frequency component of the image. The image is recovered by applying an inverse Fourier transform to the sampled k-space.

Two design quantities follow directly from this sampling geometry. The k-space sample spacing determines the field of view: $\mathrm{FOV} = 1/\Delta k$. If the object extends beyond this FOV, it aliases back into the image. The k-space extent determines the nominal pixel size, $\Delta x = \mathrm{FOV}/N \approx 1/(2k_{\max})$. Acquiring a wider k-space window improves spatial resolution, but it requires more samples. In particular, higher k_y resolution means acquiring more phase-encode rows, which usually requires additional sequence repetitions.

FFT ordering. The Fourier relationship above defines the k-space/image-space mapping, but not the array ordering used to store samples. KomaMRI returns Cartesian ADC data in acquisition order, with the DC sample $S(0,0)$ near the centre of the stacked k-space matrix. For an even matrix size N, the standard acquisition convention samples one extra negative line,

$$-N/2,\ -N/2+1,\ \ldots,\ 0,\ \ldots,\ N/2-1,$$

so for $N = 32$ this is $-16, \ldots, 0, \ldots, 15$. DC therefore lies at index $N/2+1$ in one-based indexing. Standard FFT routines instead expect DC at the first array index, so reconstruction requires

$$I = \mathrm{fftshift}\left(\mathrm{ifft}\left(\mathrm{ifftshift}(S)\right)\right),$$

Here ifftshift converts the centre-DC acquisition ordering to FFT ordering, and fftshift recentres the reconstructed image. Without these shifts, the image is reconstructed under the wrong indexing convention, causing a half-FOV shift and possible checkerboard phase modulation.

2.1.4 Gibbs (Truncation) Ringing

Finite k-space support. The Fourier relationship is exact only if all of k-space is measured. In practice, the acquisition covers a finite $N_{pe} \times N_{fe}$ rectangle. This is equivalent to multiplying the ideal infinite k-space by a rectangular window. By the convolution theorem, multiplication in k-space becomes convolution in image space:

$$I_{\text{recon}}(\mathbf{r}) = I_{\text{true}}(\mathbf{r}) * \text{PSF}(\mathbf{r}). \tag{2.8}$$

The point-spread function (PSF) is the Fourier transform of the sampling window. For a one-dimensional rectangular window, it is the Dirichlet kernel,

$$\text{PSF}(x) = \frac{\sin(\pi N x/\text{FOV})}{N \sin(\pi x/\text{FOV})}. \tag{2.9}$$

This PSF has a narrow main lobe but slowly decaying side-lobes. At sharp intensity transitions, such as the water–sphere boundaries in the phantom, those side-lobes do not cancel and appear as Gibbs ringing. Increasing N makes the ringing finer in physical units, but it does not remove the side-lobes; the artefact is caused by the rectangular truncation itself. The usual suppression method is apodisation, such as a Hamming window, which down-weights outer k-space to reduce ringing at the cost of a wider PSF and therefore slightly lower spatial resolution. Appendix A.5 derives this trade-off, since it is used directly by the phantom library's Hamming reconstruction option.

2.1.5 Pulse Sequences

A pulse sequence is a description of a series of RF pulses and MR-signal acquisitions. These are "hand-crafted by MR experts, who combined strong intuitions and an understanding of spin physics with mathematical solutions of the Bloch equations"[5]. They try to maximise signal-to-noise ratio (SNR) and image contrast while reducing scan times. To do this, they optimise parameters such as the echo time (TE) and repetition time (TR).

For example, the time between two RF pulses is called the repetition time.

2.1.6 MRI Simulators & Pulseq

Pulseq is a standardised, open-source framework for defining MRI pulse sequences. Pulseq is hardware-agnostic (compatible with all MR-Linacs).[9]. Pulseq can also be used with many MRI simulators, including both KomaMRI and MRZero.

KomaMRI is a high-performance Julia-based MRI simulator which enables scalable, multi-threaded computation to speed up simulations. [10] MRZero is a "PyTorch-based framework for easy MRI sequence optimisation and development of self-learning sequence development strategies". [11]

2.1.7 Magnetic Resonance Fingerprinting (MRF)

Conventional MRI predominantly produces qualitative, “weighted” images where pixel intensity represents a relative contrast. This depends on acquisition parameters such as coil sensitivity, and makes it difficult to standardise across different scanners. In contrast, Quantitative MRI (qMRI) aims to measure the fundamental physical properties of tissues, such as the longitudinal (T_1) and transverse (T_2) relaxation times mentioned earlier. A significant advancement in this field is Magnetic Resonance Fingerprinting (MRF), which replaces traditional steady-state sequences with ”pseudorandomized acquisition parameters”. By varying the flip angles and repetition times (T_R) throughout the scan, MRF generates a unique signal evolution—or “fingerprint”—for every tissue type. These signals are then pattern-matched against a pre-computed dictionary of Bloch equation simulations, allowing for the simultaneous and rapid mapping of multiple tissue properties in a single acquisition. For MR-guided radiotherapy, this quantitative approach offers the potential for “biological gating”, where treatment adaptation is driven by functional changes in tumour physiology rather than just anatomical shifts. [12] [13]

2.2 Magnetic Resonance Linear Accelerator (MR-LINAC)

The MR-LINAC represents a paradigm shift in radiation oncology, moving beyond conventional IGRT, which provides only a pre-treatment ’snapshot’ of the patient’s anatomy. Instead, it enables a continuous feedback loop between real-time imaging and dose delivery, allowing for dynamic adjustment during the treatment itself, known as Adaptive Radiotherapy (ART).

2.2.1 Clinical Benefits of ART

By capturing high-contrast images while the patient is on the treatment table, clinicians can account for ”inter-fraction” changes (variations between daily sessions) and ”intra-fraction” motion (movement during the session, such as breathing or bowel movements).

This real-time visualisation allows for a significant reduction in the Planning Target Volume (PTV). Traditionally, large margins are added to the Clinical Target Volume (CTV) to compensate for uncertainties in organ position. Using MRgRT, these margins can be tightened. In prostate cancer, PTV margins have been reduced to 2mm, significantly reducing the volume of radiated healthy tissue. Further, better soft tissue contrast is essential to accurately delineate the tumour and contour Organs at Risk (OaR) in complex regions like the liver and pancreas.

This allows for ”hypofractionation” (delivering a more potent dose over fewer sessions) without increasing the toxicity profile for the patient. This significantly decreases treatment side effects. [14]

2.2.2 Current Limitations and Implementation Challenges

Despite its potential, the global adoption of MR-LINAC technology remains in its early stages. As of 2024, four primary operational configurations exist, typically utilising magnetic field strengths of either 0.35T (e.g., ViewRay MRIdian) or 1.5T (e.g., Elekta Unity) [14]. The transition to MRgRT introduces several operational bottlenecks:

- **Complexity and Workflow:** The requirement for “on-table” plan adaptation demands a high level of expertise and significant upskilling for radiation therapists, physicists, and oncologists.
- **Delineation Uncertainty:** While tumours frequently shrink during treatment, it is currently unclear if adapting to these smaller volumes is always safe, as the original tumour bed may still contain microscopic disease [4].
- **Time Constraints:** Re-tuning treatment plans while the patient is immobilised requires rapid image acquisition and contouring to prevent motion artifacts and patient discomfort.

2.2.3 The Role of AI and Quantitative MRI

To address the clinical bottleneck of manual contouring, AI-based automated segmentation has emerged as a vital tool. Tools such as the Philips RTdrive Core have demonstrated clinical robustness in prostate cancer patients, significantly reducing the workload for clinicians by automatically outlining the target and OaRs. [15].

Furthermore, the future of MRgRT lies in moving beyond simple anatomical contrast. The integration of quantitative MRI (qMRI), such as measuring tissue cell density through Diffusion-Weighted Imaging, allows for biological targeting. By applying Deep Learning and Reinforcement Learning (RL) to these multi-parametric datasets, the MR-LINAC could eventually provide an end-to-end automated system that optimises dose delivery based on the functional response of the tumour in real-time [4].

2.3 Deep Learning in MRI

Long acquisition times represent a major barrier in clinical MRI, primarily due to the inherent trade-off between image quality and speed of imaging. Indeed, the 3D images crucial for tumour delineation

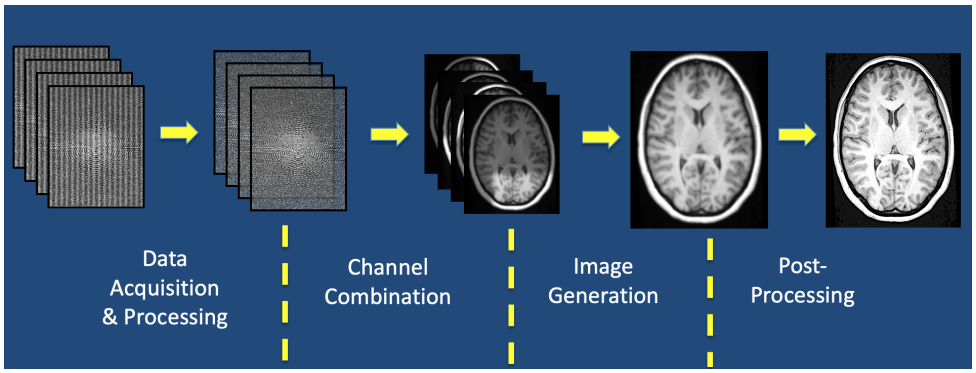


Figure 2.6: Courtesy of Allen D. Elster, MRIquestions.com. [16]

require longer acquisition times, which increases the risk of motion artifacts. There is a large body of research into reducing acquisition time while maintaining image quality. Notable breakthroughs include Parallel Imaging (multiple receiver coil arrays) and Compressed Sensing. Unfortunately, Parallel Imaging quickly reaches practical Signal-to-Noise Ratio (SNR) limits, while compressed sensing is prohibitively computationally expensive. [5]

To supplement, DL has been widely applied, both in research and through commercial products, to reduce imaging time and improve image quality. For example, DL is used in post-processing to remove noise, improve image resolution and reduce image inaccuracies (called artifacts) from patient movement or field inhomogenities (e.g. caused by metal patient implants or pacemakers). [16]

2.3.1 Image Acquisition and Reconstruction using Deep Learning

Post-processing operates only on the output image, which allows leveraging traditional imaging techniques (CNN, image kernels) as well as being vendor-neutral (so hardware-agnostic). However, not having the raw signal limits the information available.

This has given rise to more complex networks that replace both the Fast Fourier Transform algorithm used for reconstruction and post-processing to create higher quality images.

2.3.2 Pulse sequence design

Recent research has explored the use of artificial intelligence to optimise the underlying physics of MR acquisition through the autonomous design of pulse sequences. Two differing approaches have been tested: an RL game-based approach and a more traditional supervised learning approach.

The MRzero framework introduces a supervised learning approach that treats the entire scanning and reconstruction pipeline as a fully differentiable system. By simulating RF events, gradient moments,

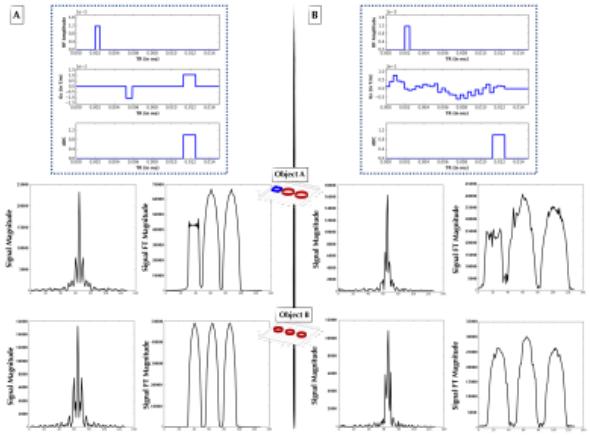


Figure 2.7: (B) Novel pulse sequence successfully developed by AUTOSEQ to mimic a gradient echo sequence. Figure from [17]

and delay times through a Bloch equation layer, the model can be optimised directly against a target contrast of interest (e.g. a T_1 contrast map). The cost function that was optimised by the model took into account: data fidelity, Specific Absorption Rate (SAR) penalties and scan time. The researchers successfully demonstrated that MRzero could "learn" to replicate conventional gradient-echo sequences from scratch. This was then successfully tested on a clinical 3T scanner for in vivo human brain imaging. This demonstrates that deep learning-derived sequences are clinically viable and can be optimised for specific hardware constraints.[11]

By recasting pulse sequence development as a "game of perfect information," researchers have implemented a reinforcement learning (RL) framework where an AI agent interacts with a Bloch equation-based physics simulator to generate novel sequence actions. This approach utilises a Bayesian derivative of RL, employing Gaussian processes and Bayesian neural networks to navigate the non-linear dynamics of nuclear magnetisation and maximise an acquisition function based on image reconstruction accuracy. Preliminary 1-D experiments demonstrate that such agents can independently "discover" canonical sequences, such as the gradient echo, and generate novel pulse sequences that approximate Fourier spatial encoding. While this was a 1-D experiment, further exploration could yield novel pulse sequences that reduce acquisition time and maximise SNR, alleviating the current acquisition bottleneck faced by the MR-Linac.[17]

2.3.3 Adaptive MRI acquisition

The above work focuses on the design of novel, but ultimately static pulse sequences. Instead, Adaptive MR is a promising new research direction. Here, a probability distribution of the tissue parameter of interest (e.g. T_2) is continuously updated with each new reading. The next pulse sequence is chosen

based on the estimate of T_2. This has been shown to accelerate acquisition time by a factor of 2.5. This is a general paradigm shift in pulse sequence design that motivates and validates further exploration of adaptive MR.[18]

Adaptive MR also has the potential to be combined with RL to autonomously control in real-time MRI hardware. Walker-Samuel demonstrated this by using a Deep Deterministic Policy Gradient (DDPG) algorithm to control an MRI simulation environment where the acquisition process had been turned into a reward-based game. The RL agent was tasked with identifying phantom shapes while being penalised for increased acquisition time. This forces the system to optimise the trade-off between Signal-to-Noise Ratio (SNR) and image acquisition time. Notably, the agent autonomously learned to perform "active sensing," acquiring only four to six outer lines of k-space and dynamically adjusting T_E, T_R, and flip angles to act as an edge detector. With a reported shape-classification accuracy of 99.8%, this research suggests that RL-controlled scanners could move beyond human-readable imaging to perform high-speed, task-specific "autonomous sensing". However, this was purely simulation-based and a very simple task, whether this method can be applied to more complex situations is unproven.

2.3.4 Reinforcement Learning and PPO for Adaptive Acquisition

Reinforcement learning (RL) studies problems where an agent improves its behaviour by interacting with an environment. At each step the agent observes the current state of the environment, chooses an action, and receives a scalar reward. The objective is not to predict a labelled target directly, as in supervised learning, but to learn a policy that selects actions leading to high cumulative reward.[19]

The standard mathematical model is a Markov decision process (MDP),

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \rho_0, \gamma), \tag{2.10}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s_{t+1} \mid s_t, a_t)$ is the transition model, R is the reward function, ρ_0 is the initial-state distribution, and $\gamma \in [0, 1]$ is the discount factor. A trajectory is a sequence

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots). \tag{2.11}$$

In this project, the environment is the simulated adaptive qMRI acquisition loop. A state contains the information available to the agent so far, such as the current acquisition history and the current estimate of the phantom parameters. An action specifies the next acquisition choice, for example timing

and RF parameters such as inversion time, echo time, repetition time, and flip angle. The simulator then returns reconstructed measurements and a reward based on whether the new acquisition improves the quantitative parameter estimate, typically measured through reduction in T_1 error.

For a stochastic policy $\pi_\theta(a \mid s)$ with parameters θ, the return from time t is

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}. \tag{2.12}$$

The RL objective is to choose policy parameters that maximise expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G_0]. \tag{2.13}$$

The expectation is over trajectories generated by the current policy interacting with the environment. This is important in simulation-in-the-loop MRI: the training data are not a fixed dataset. They are samples produced by repeatedly running the current policy, simulating the resulting sequence, reconstructing the image, fitting the tissue parameters, and computing the reward.

Policy gradients

Policy-gradient methods optimise $J(\theta)$ directly.[20], [21] The probability of a trajectory under a policy is

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1} \mid s_t, a_t)\pi_\theta(a_t \mid s_t). \tag{2.14}$$

The simulator dynamics P do not depend on the policy parameters. Applying the log-derivative trick gives the basic policy-gradient expression:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t\right]. \tag{2.15}$$

In practice this expectation is approximated using sampled rollouts:

$$\hat{g} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T_i-1} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \hat{A}_{i,t}. \tag{2.16}$$

This is the central sample-based approximation of policy-gradient RL: the expectation over all trajectories is estimated from a finite batch collected from the simulator, making actions with better-than-expected outcomes more likely and worse-than-expected actions less likely. Crucially, it does not require

differentiating the simulator. The gradient flows only through the policy's log-probability $\log \pi_\theta(a_t \mid s_t)$, which is differentiable because the policy is a neural network, so KomaMRI can stay a black-box physics simulator used only to sample transitions and rewards. The weighting term is written as an advantage estimate $\hat{A}_t$ rather than the raw return G_t, which lowers variance by measuring whether an action beat what the policy already expected in that state.

Value and advantage functions

The value function of a policy is the expected future return from a state:

$$V^\pi(s) = \mathbb{E}_{\tau\sim\pi}[G_t \mid s_t = s]. \tag{2.17}$$

The action-value function is the expected return after taking a particular action and then continuing with the policy:

$$Q^\pi(s,a) = \mathbb{E}_{\tau\sim\pi}[G_t \mid s_t = s, a_t = a]. \tag{2.18}$$

Equivalently, it can be written in one-step form as the expected immediate reward plus the value of the next state:

$$Q^\pi(s_t,a_t) = \mathbb{E}\left[r_t + \gamma V^\pi(s_{t+1}) \mid s_t, a_t\right]. \tag{2.19}$$

The advantage function compares these two quantities:

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s). \tag{2.20}$$

An action with positive advantage performed better than the policy's average action at that state; an action with negative advantage performed worse. This is the quantity used to decide which sampled actions should become more or less probable.

In deep RL we usually do not know V^π, Q^π, or A^π exactly. PPO therefore learns two approximations at the same time:

- an actor, $\pi_\theta(a \mid s)$, which chooses acquisition actions;
- a critic, $V_\phi(s)$, which predicts the expected return from the current state.

The critic provides a baseline for the policy-gradient update. A simple one-step temporal-difference residual is formed from an actual sampled transition. At time t, the actor samples an action $a_t \sim \pi_\theta(\cdot \mid s_t)$; the simulator applies this acquisition action and returns the immediate reward r_t and the next state s_{t+1}. The residual is then

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t). \tag{2.21}$$

Thus $V_\phi(s_t)$ is the critic's prediction before taking the action, while $r_t + \gamma V_\phi(s_{t+1})$ is the one-step target after seeing the simulator's response to the sampled action.

Generalised Advantage Estimation (GAE) combines these residuals over multiple future steps:[22]

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \tag{2.22}$$

where λ controls the bias-variance trade-off. In words, the value network learns what return is expected from a state, and the advantage estimate measures whether the sampled action improved on that expectation. For this project, that means the critic learns to predict future improvement in the quantitative MRI objective, while the actor learns which sequence parameters tend to produce those improvements.

GAE thus interpolates between two estimates of the same advantage $A^\pi(s_t, a_t)$, computed from rollouts whose rewards are already known: the full Monte Carlo return ($\lambda \to 1$) is unbiased but high-variance because it leans on one noisy trajectory, while the one-step residual ($\lambda \to 0$) leans on the learned critic, cutting variance at the cost of bias when the critic is inaccurate.

Proximal policy optimisation

Vanilla policy-gradient updates can be unstable because a single gradient step can move the policy too far. If the new policy differs greatly from the policy that generated the rollout batch, the sample estimate may no longer describe the behaviour of the updated policy. Trust Region Policy Optimisation (TRPO) addresses this by constraining the KL divergence between the old and new policies, but it requires a more complex second-order optimisation procedure.[23]

The underlying update is an update to the neural-network weights θ. Even if the parameter change looks small in weight space, it can cause a much larger change in the induced probability distribution over actions, $\pi_\theta(\cdot \mid s)$. Since PPO is on-policy, the rollout batch was collected using the old policy. If

the update makes the new policy assign very different action probabilities, the batch becomes stale: it reflects what was useful under the old action distribution, not necessarily what is useful under the new one. Large updates can therefore overreact to lucky sampled actions or collapse exploration.

Proximal Policy Optimisation (PPO) keeps the same basic idea – improve the policy while preventing large destructive updates – but uses a simpler clipped objective.[24] For samples collected with the old policy $\pi_{\theta_{\text{old}}}$, define the likelihood ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}. \tag{2.23}$$

If $r_t(\theta) > 1$, the new policy assigns higher probability to the sampled action than the old policy did. If $r_t(\theta) < 1$, it assigns lower probability. PPO maximises

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t \right) \right]. \tag{2.24}$$

For positive-advantage actions, the objective encourages increasing their probability, but only up to the clipping threshold. For negative-advantage actions, it encourages decreasing their probability, again only up to the threshold. The clipping term therefore removes the incentive for the policy to move far away from the sampled policy in a single update.

The full optimisation also trains the value function and usually includes an entropy bonus to preserve exploration:

$$L(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_v \mathbb{E}_t \left[\left(V_\phi(s_t) - \hat{G}_t \right)^2 \right] - c_H \mathbb{E}_t \left[\mathcal{H}(\pi_\theta(\cdot \mid s_t)) \right]. \tag{2.25}$$

Here c_v and c_H weight the critic loss and entropy bonus. The practical loop is to collect rollouts with the current policy, estimate returns and advantages, update the actor and critic for a few minibatch epochs, then discard the rollouts and collect fresh ones. This makes PPO *on-policy*: unlike off-policy methods such as DQN or SAC, which keep a *replay buffer* of past transitions and reuse them many times, PPO trains only on data from the current policy and throws it away after each update. That costs sample efficiency but avoids the instability of learning from a stale action distribution, and it removes the replay buffer's memory overhead.

PPO suits adaptive qMRI for three reasons: it handles continuous action spaces (the sequence timings), it explores stochastically while the clipped objective prevents sudden policy collapse, and its on-policy simplicity makes the physics-and-reward path easier to debug than an off-policy learner. The cost

is sample efficiency: every update needs fresh rollouts, and one rollout step here is not a cheap game transition but a full MR sequence build, KomaMRI Bloch simulation, image reconstruction, ROI extraction, T_1 fit, and reward evaluation. The number of simulator calls is therefore the dominant training bottleneck, which is what motivates the multi-fidelity training loop of Chapter 5.

Recurrent policies for partial observability

The PPO actor and critic above are feed-forward networks that map the current observation s_t to an action distribution and a value. This implicitly assumes s_t is a sufficient statistic of the history, i.e. the Markov property holds. When the observation does not capture everything relevant about the past, the problem is a partially observed MDP (POMDP) and a memoryless policy can be suboptimal. Adaptive qMRI is partially observed in exactly this way: a compact observation of the current fitted estimates and remaining budget does not record which acquisition timings have already been played, yet two histories with the same current fit can carry different information for the next block.

A recurrent policy adds an internal memory. The feed-forward trunk is preceded by a recurrent network, here a Long Short-Term Memory (LSTM) cell [25], that maintains a hidden state h_t updated each step from the previous state and the current observation, $h_t = \mathrm{LSTM}(h_{t-1}, s_t)$. The action and value are conditioned on h_t rather than s_t alone, so the policy can summarise the acquisition history into h_t and condition the next timing on it; the LSTM's gating lets it retain information over many steps without the vanishing gradients of a plain recurrent network. Training uses the recurrent variant of PPO, identical to the above except that rollouts store hidden states and gradients are propagated back through each rollout's recurrent steps (truncated backpropagation through time). Chapter 5 evaluates such a policy as an ablation against the compact feed-forward observation, testing whether explicit memory of the acquisition history improves adaptive timing.

2.3.5 Clinical Implications

This research landscape is the early phases of a transition from human-engineered MRI protocols to a paradigm of "autonomous radiology." While the MR-LINAC offers unparalleled soft-tissue visualisation, its clinical utility is currently throttled by the latency of traditional imaging and manual contouring. The integration of Bayesian reinforcement learning [14], end-to-end supervised learning [11], and task-specific autonomous control [26] could provide a technical pathway to overcome these limitations. By treating the MR-LINAC not as a static imaging tool, but as a dynamic, AI-controlled sensor, it becomes possible to collapse the "imaging-planning-delivery" cycle into a single, synchronised event. This evolution toward an automated, closed-loop system will likely be the catalyst that allows MR-guided radiotherapy to achieve

its full potential, providing real-time, biologically adaptive treatments that respond to tumour dynamics.

3

MRISystemPhantom.jl — Digital Twin Library

Contents

3.1 The physical phantom

The QalibreMD NIST/ISMRM System Standard Model 130 (manufactured by Caliber MRI, formerly QalibreMD) is a hardware phantom designed to mimic the geometry of a human head and to provide a wide, calibrated range of quantitative MRI tissue parameters (T1, T2, and proton density). It is the standard reference device for quantitative MRI system validation in clinical and research settings, and its design is described in the *QalibreMD System Standard Model 130 User Manual* (available here).

The phantom consists of a spherical water-filled housing (100 mm internal radius) containing four internal structures at fixed axial positions:

Plate	z (mm)	Contents
T1 array	+56.5	14 $NiCl_2$ spheres (15 mm radius), T1 range ~24 ms – 1.9 s at 3 T
T2 array	+16.5	14 $MnCl_2$ spheres (15 mm radius), T2 range ~87 ms – 2.8 s at 3 T
PD array	-23.5	14 spheres of varying proton density (15 mm radius)
Fiducial grid	distributed	57 small spheres (5 mm radius) on a 40 mm cubic lattice

Each contrast plate arranges its 14 spheres in two concentric rings: 10 on an outer ring of radius 65 mm and 4 on an inner ring of radius 28 mm. This geometry matches the phantom manual photographs but has not been verified against the physical device. The current digital twin models the contrast arrays and fiducial grid, but not the resolution insets or the two slice-profile wedges.

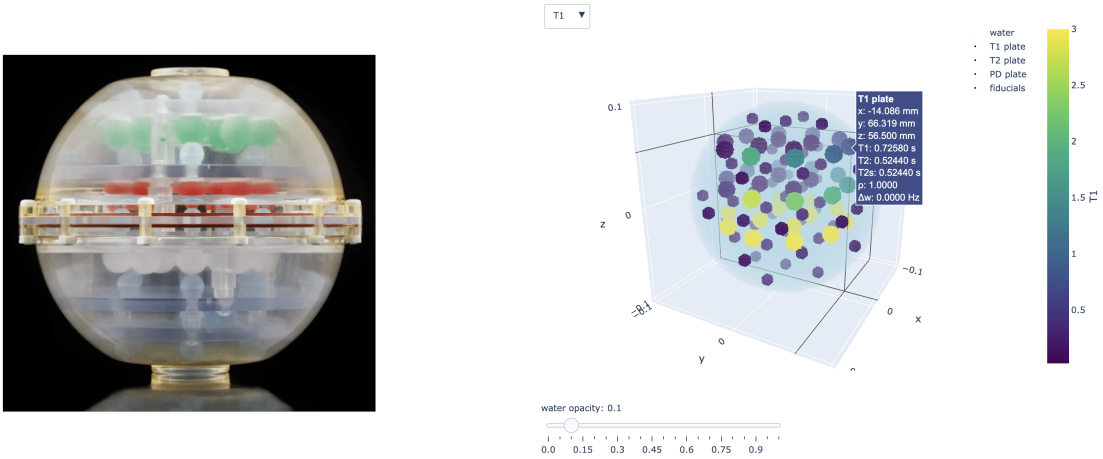


Figure 3.1: **The QalibreMD NIST/ISMRM System Standard Model 130 hardware phantom and its digital T1 array.** The physical phantom photo (left) is reproduced from Caliber MRI. The voxelised plot of the digital phantom (right), coloured by T1 value, can be reproduced by running `examples/plot_phantom_3d.jl` in MRISystemPhantom.

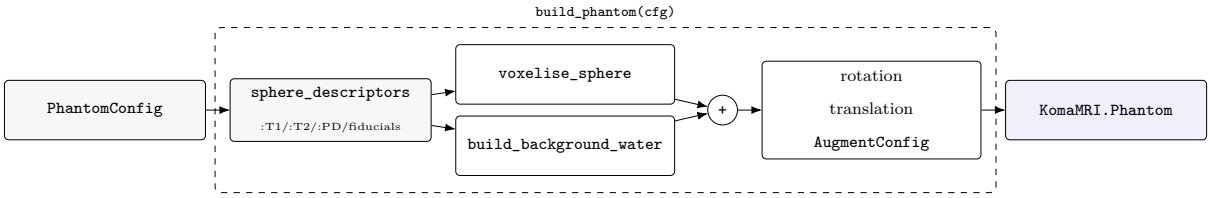
The relaxation values are field-strength-dependent, so separate reference values are provided for 1.5 T and 3 T. The user manual also distinguishes between legacy and modern serial-number classes; both sets of reference values are included in the library.

3.2 Design goals and architecture

A digital twin of this phantom is needed for two reasons. First, it provides a ground-truth environment for RL training: the agent needs to query a simulator that returns physically meaningful signals and can report the true tissue parameters for reward computation. Second, it must be **flexible**: the RL training loop needs to randomise pose, inject noise, and jitter reference values across episodes without modifying library code. Different experiments also require different phantom sections or resolution.

The library is structured around a single contract: a `PhantomConfig` struct that carries the builder configuration. A single function, `build_phantom`, returns a `KomaMRI.Phantom`, which can then be fed directly to `KomaMRI.simulate`. This design was chosen deliberately over a mutable phantom object or a collection of keyword arguments as the config is serialisable, diffable, and reproducible (the RNG seed is embedded), and the builder has no hidden state. This is useful for comparing different RL runs and also simplifies the Python-Julia boundary used by the training code.

The pipeline from `PhantomConfig` to `KomaMRI.simulate()` is:



The two-stage design — descriptors first, voxels second — means all geometry and material decisions happen before any memory is allocated for spin arrays. Descriptors are also individually modifiable and removable: you can simulate only a subset of the 14 spheres per plate, with the water housing automatically filling the vacated volume. Because each descriptor carries its physical centre coordinate, `sphere_descriptor_pixels` can convert those centres directly to pixel indices in the reconstructed image — using the same DC-at-centre convention as `kspace_to_image` — simplifying ROI extraction after reconstruction.

3.3 Geometry

The geometry implementation is judged by whether it preserves the reconstructed sphere measurements, not by whether every background-water spin is represented exactly.

3.3.1 Sphere voxelisation

Each sphere is voxelised on a cubic grid at spacing `delta_x = voxel_size_mm * 1e-3` metres. A spin is included if its grid point falls within the sphere radius:

$$(x_i - c_x)^2 + (y_i - c_y)^2 + (z_i - c_z)^2 \leq r^2$$

The grid is constructed by iterating over the bounding box `[c - r, c + r]` on each axis. For a typical 15 mm radius sphere at 2 mm voxels this produces around 500 spins per sphere. Across all three contrast plates (42 spheres), the combined spin count at 2 mm voxels is ~9,100, occupying ~570 KB (8 Float64 fields per spin: $x, y, z, \rho, T_1, T_2, T_2^*, \Delta\omega$), small enough that the full set fits comfortably in memory and the per-step simulate call is fast.

3.3.2 Slicing

For 2D imaging experiments, only spins within a thin axial slab need to be simulated, reducing the spin count, and therefore simulation time, substantially. A `Slab` is defined by a centre point $\mathbf{c}$, a unit normal $\hat{n}$, and a thickness t. A spin at position $\mathbf{p}$ is included if its signed distance from the midplane satisfies:

$$|(\mathbf{p} - \mathbf{c}) \cdot \hat{n}| \leq t/2$$

The normal defaults to $\hat{z}$ but any orientation is supported. A 0.1 μm tolerance is applied at the boundary to avoid floating-point rejections of on-edge voxels. At 1 mm voxels, a 5 mm axial slab centred on the T1 plate retains ~33,000 of the full phantom's ~4.2 million spins, a 128× **reduction**. This matches the excited volume of a slice-selective RF pulse without needing to encode one explicitly.

3.3.3 Pose transforms

`apply_transform!` rotates and translates all spin positions by the Euler XYZ rotation and translation specified in `PhantomConfig.rotation` and `translation_mm`. The rotation matrix is built from three successive Rx, Ry, Rz multiplications. The transform is applied after voxelisation and before augmentation noise so that the sphere geometry is exact before any stochastic perturbation. This was designed to make it easy to reset the phantom and reduce overfitting.

3.3.4 Background water

The housing is voxelised as a 100 mm-radius sphere with `water_voxel_size_mm` spacing (defaulting to `voxel_size_mm`), with all contrast and fiducial sphere volumes cut out by the negated form of the squared-distance test used for voxelisation. The water housing is by far the largest spin population, yet it is not a quantity the phantom exists to measure, so it is the natural place to trade fidelity for simulation speed. We do this by increasing `water_voxel_size_mm` to coarsen the grid, thus reducing the spin count. We then reweight each spin's proton density by `(water_dx / sphere_dx)`^{3}. This conserves the total water magnetisation, so the bulk water signal amplitude is correct regardless of the coarsening factor.

This coarsening must also handle anisotropic resolution as slices are typically thick (3–8 mm) relative to the in-plane resolution (1–2 mm). So, when a slice is selected the housing is projected onto a grid of stacked plane-sheets (`water_throughplane_voxel_size_mm` apart). Again, these are re-weighted to conserve total magnetisation.

The figure below quantifies the fidelity–cost trade for an IR-SE-2D acquisition.

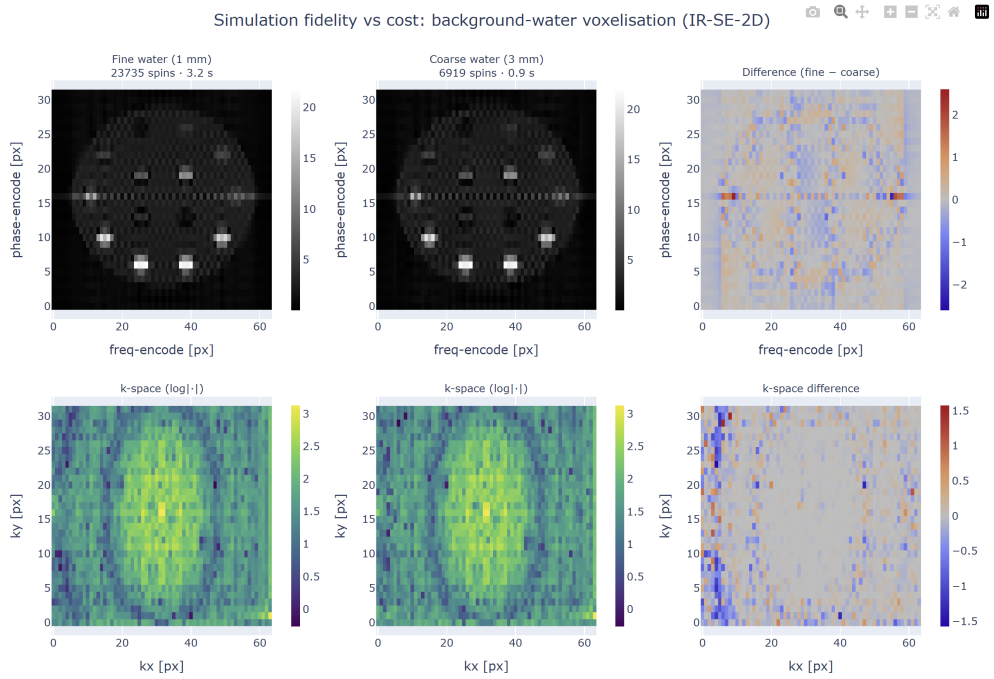


Figure 3.2: **Fidelity–cost trade-off of background-water voxelisation.** The same IR-SE-2D acquisition (TI/TE/TR = 400/80/1500 ms, 32 × 64, FOV 0.2 m) is simulated through two phantoms differing *only* in background-water discretisation. **Top row:** magnitude images; **bottom row:** log-magnitude k-space; **right column:** fine - coarse difference. The figure and numbers are reproduced by `examples/compare_water_coarseness.jl`.

Conserving the bulk water signal is necessary but not sufficient. The k-space DC term changes by only 0.35%, but the phantom is used to measure the sphere signals. The coarse 3 mm grid uses **6,919 vs 23,735 spins** (3.4× fewer) and is ~3.4× **faster to simulate** (0.9 ± 0.1 s vs 3.2 ± 0.6 s, mean ± std over 20 runs). Using the `sphere_descriptor_pixels` mapping, the single centre-pixel coarse-vs-fine NRMSE

at the 32×64 matrix is **3.7%**.

The larger single-pixel error reflects where the coarse grid differs from the fine one. Reweighting preserves the amount of water, but not the exact water/sphere boundary: the 3 mm water grid stair-steps the sphere cut-outs, changing high-spatial-frequency edge content while leaving low-frequency k-space essentially unchanged. The boundary error is on the scale of the reconstructed pixels here: for a 200 mm FOV and 32×64 matrix, the spacing is 6.25 mm in phase encode and 3.125 mm in frequency encode. After band-limited reconstruction, this boundary mismatch appears as a slightly different Gibbs-ringing pattern around the spheres.

Most of this error is concentrated in the near-null spheres at this inversion time, especially T1-spheres 13-14, whose signals move by about 9%, while the bright spheres change by $\leq 0.7\%$. This is expected because the ringing leakage is approximately additive, set mostly by the surrounding bright water.

If the sphere signal is measured as a small central ROI rather than a single point sample, the error falls to **1.5%** for a 3×3 ROI. This is the more relevant operational metric for an ROI-based phantom readout; the single-pixel value remains useful as a worst-case sensitivity measure because it exposes pixel-phase effects, Gibbs oscillation, and sub-pixel centre rounding.

The same pattern strengthens at 64×64: with unchanged spin counts and DC term, the errors rise to **7.6%** for a single centre pixel and **2.3%** for the 3×3 ROI, consistent with a wider k-space window admitting more of the edge mismatch.

Because the discrepancy is carried by truncation side-lobes, it can be suppressed at reconstruction. The library provides a 2-D Hamming window through `hamming_window_2d` and `kspace_to_image(...; hamming=true)`. As derived in the background chapter, this reduces the PSF side-lobes from -13 dB to -43 dB, at the cost of a wider main lobe.

Reconstructing both grids with the Hamming window reduces the single-pixel coarse-vs-fine NRMSE by four-fold, from **3.7% to 0.9%**. This confirms that the single-pixel discrepancy was dominated by truncation ringing. The 3×3-ROI error improves more modestly, from **1.5% to 1.1%**, because ROI averaging already cancels much of the oscillatory side-lobe structure. Apodisation and spatial averaging therefore suppress largely the same error component, rather than providing independent gains. The remaining $\approx 1.1\%$ is the residual coarse-grid discrepancy under this ROI-based measurement protocol.

The Hamming window down-weights outer k-space, visible as the darkened periphery of the centre k-space panel in Figure 3.3. This widens the PSF main lobe and slightly blurs the spheres, but suppresses ringing side-lobes. The difference is localised to sphere edges and high-frequency k-space, consistent with suppression of truncation ringing rather than a change in the smooth low-frequency signal. The window

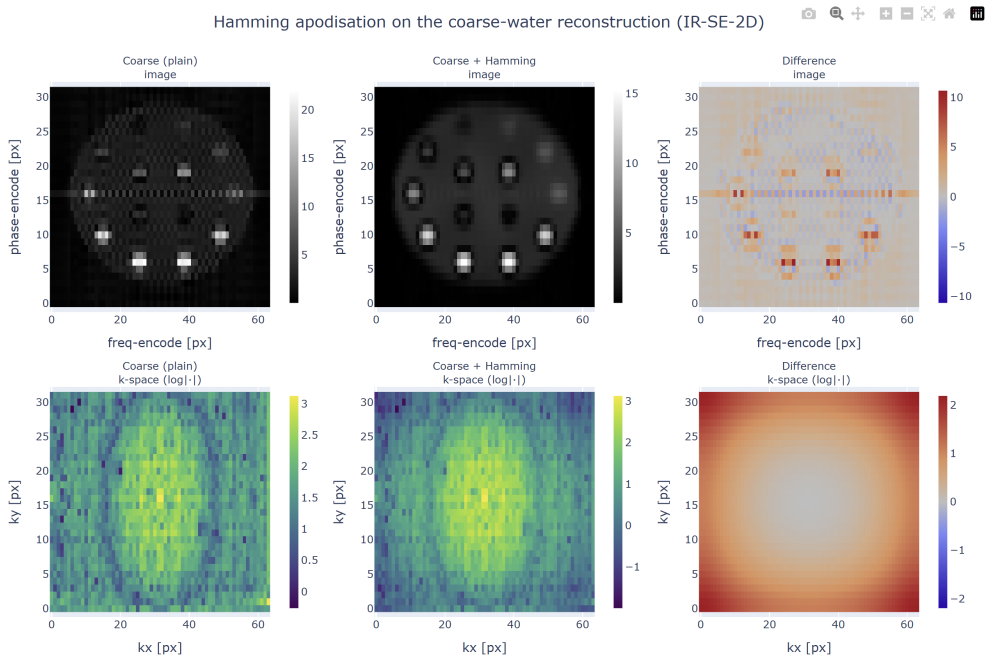


Figure 3.3: **Effect of Hamming apodisation on one reconstruction.** A coarse-water phantom reconstructed without a Hamming window (left), with a 2-D Hamming window (centre), and their difference (right). Produced by `examples/hamming_apodisation.jl`.

is a reconstruction-side element-wise multiplication and costs only a fraction of a millisecond, so it does not affect the roughly 3× simulation speed-up from the coarse grid. Its cost is resolution, through the wider PSF main lobe.

A natural future improvement is a boundary-aware water grid: keep the smooth water bulk coarse, but voxelise a thin shell around the housing surface and sphere cut-outs at fine resolution so that coarsening no longer changes the boundary geometry.

3.4 Material tables

The relaxation values are read from four module-level constants defined from the phantom manual:

Constant	Spheres	Source
`T1_ARRAY[:T3]` / `[:T15]`	14 T1-plate spheres	$NiCl_2$ recipe, manual table, Serial Number $\geq$ 0042
`T1_ARRAY_LEGACY[:T3]` / `[:T15]`	same spheres	manual table, SN pre-0042 (different values)
`T2_OF_T1_ARRAY`	T2 companions for T1 plate	same manual table
`T2_ARRAY[:T3]` / `[:T15]`	14 T2-plate spheres	$MnCl_2$ recipe
`T1_OF_T2_ARRAY`	T1 companions for T2 plate	same manual table

Constant	Spheres	Source
`PD_FRACTIONS`	14 PD-plate spheres	proton density fractions
`BACKGROUND_WATER`	housing water	distilled water at 20°C

The T1 range at 3 T spans two decades: 23 ms to 1.84 s for the $NiCl_2$ plate, covering the full range of biological tissue T1 values from white matter to cerebrospinal fluid. The T2 range spans ~87 ms to 2.76 s. These wide ranges are why the phantom is used for system validation and make it useful for RL training: to ensure the learned policies can produce reliable estimates across the full physiological range.

The `serial_number_class` field switches the T1 table between current and legacy recipes. All reference values are currently fixed at 20°C; the manual includes temperature-correction graphs, but these are not yet implemented.

Accuracy caveats. Two entries are approximate: PD-plate T1/T2 values (`pd_t1`, `pd_t2`) use bulk water regardless of proton-density fraction (the manual does not tabulate per-sphere relaxation for the PD plate), and fiducial relaxation values (`FIDUCIAL_PROPS`) are generic $CuSO_4$ placeholders not verified against any calibration record. Neither affects the T1- or T2-plate experiments, but both should be replaced before any simulation-to-real comparison involving the PD plate or fiducial grid.

3.5 Augmentation

`AugmentConfig` enables domain randomisation for RL training. All fields default to zero. When non-zero values are provided, `apply_per_spin_noise!` draws from the relevant distributions using the config's `rng_seed`:

Field	Effect
`T1_sigma_rel`	Multiplies each sphere's T1 by `exp(sigma*z)` where `z ~ N(0,1)`, drawn once per sphere
`T2_sigma_rel`	Same for T2
`PD_sigma_abs`	Adds `N(0, sigma^2)` to each spin's proton density
`position_sigma_mm`	Adds `N(0, sigma^2)` to each spin's (x, y, z) position independently
`B0_sigma_Hz`	Adds `N(0, sigma^2)` per-spin off-resonance, stored in Δw (rad/s after $\times 2\pi$)

Field	Effect
`drop_sphere_p`	Drops each sphere independently with Bernoulli probability p before voxelisation

The T1/T2 jitter is applied at the sphere level (one draw per sphere, not per spin) to simulate manufacturing variability in the doping concentration. Of the fields above, only `T1_sigma_rel` and `T2_sigma_rel` were used in the experiments of this work; the remaining fields are implemented for completeness but were not exercised.

The `rng_seed` is the only source of stochasticity: fixing it makes `build_phantom` deterministic, so the same configuration always produces the same phantom. Fixing the seed reproduces the exact same phantom, making evaluation runs reproducible. Incrementing it each training episode draws a statistically independent phantom, preventing the agent from memorising a fixed configuration.

3.5.1 Episode-level random phantoms

A fixed calibrated phantom is unsuitable for RL training because the agent can exploit the known sphere ordering. Instead, each episode should draw a new plausible phantom. Earlier RL code did this manually by editing sampled `SphereDescriptor`s and assembling a fresh `PhantomConfig`. `RandomPhantomConfig` makes this explicit: it is a distribution over deterministic `PhantomConfig`s. The base config fixes build settings such as field strength, voxel size, included plates and water model; the random config specifies what changes between episodes: active contrast spheres, sampled material values and phantom pose.

This is distinct from `AugmentConfig`. `AugmentConfig` adds low-level simulation perturbations after the phantom has been voxelised, such as per-spin off-resonance or position noise. It is useful for modelling measurement imperfections, but it does not change the episode's true sphere-level material values. In contrast, `RandomPhantomConfig` samples descriptor-level properties before voxelisation. For example, one training episode might select five T1 spheres, draw one T1 value for each selected sphere from a log-normal distribution around its nominal value, set T2 by preserving the sphere's nominal T2/T1 ratio, and then apply a small in-slice rotation and translation. `sample_phantom_config` returns both the sampled `PhantomConfig`, which is safe to pass to `build_phantom`, and a hidden truth record used for rewards and evaluation.

The pose sampler is experiment-dependent. Thin-slice acquisitions use only an in-plane rotation and x/y translation, because a full 3D rotation would move spheres out of the acquired slab. Volumetric experiments can instead use `UniformSO3PoseSampler`, which draws a true uniform orientation on SO(3)

and stores it as a 3×3 rotation matrix. For this reason, `PhantomConfig` accepts both Euler-angle triples and precomputed rotation matrices.

Samplers are explicit objects, using `Distributions.jl` where appropriate [27], so assumptions such as material jitter, sphere dropout and pose noise are visible rather than hidden in scattered `randn` calls. Material distributions can also depend on other sampled tissue parameters: for example, T1 can be drawn from `Uniform(0.3, 2.5)` while T2 is defined by `PreserveNominalRatio(:T2, :T1)`. These property dependencies are evaluated in order and checked for cycles, so invalid specifications fail at sampling time. The API remains generic: selectors, material samplers and pose samplers operate on `SphereDescriptor`s and `PhantomConfig`s, not on RL states, observations or reward code.

3.6 Shipped sequences and fitting models

The background chapter explains what the main MRI sequence families are. This section instead documents what `MRISystemPhantom.jl` ships, why those routines are needed by the experiments, and which assumptions they encode. All sequence builders return standard `KomaMRI.Sequence` objects, so they can be passed directly to `KomaMRI.simulate`. The fitting routines are pure Julia and are shared by the conventional baselines and RL environments, so reported gains are measured against the same estimators.

Single-spin sequence helpers

- `ir_sequence(TI)`: 180° inversion, delay, 90° excitation and ADC. Used for simple T1 checks and the conventional T1 helper; after full recovery the first ADC sample follows $|1 - 2e^{-\mathrm{TI}/T_1}|$.
- `se_sequence(TE)`: 90°–180° spin echo for the T2 baseline. The echo magnitude follows $S_0 e^{-\mathrm{TE}/T_2}$.
- `mse_sequence(ESP, n_echoes)`: CPMG / multi-echo spin echo. One excitation is followed by a train of 180° pulses; echo k occurs at k ESP, so a whole T2 decay is sampled in one TR.

2D Cartesian sequence builders

- `ir_se_2d_sequence(TI, TE, TR; ...)`: main spatial T1 acquisition for E2. It is parameterised by α_{exc}, `FOV`, `Nfe` and `Npe`; it acquires one k-space row per shot, so fits pass `Npe` to account for the transient ramp from thermal equilibrium.
- `ir_tse_2d_sequence(TI, esp, TR; ...)`: turbo spin-echo variant with echo-train length `etl`. It is faster by approximately `etl` and exposes T2 information when TE varies across echoes.
- `se_2d_sequence(TE, TR; ...)`: 2D spin echo without inversion preparation, used for spatial T2 mapping by sweeping TE.

- `gre_2d_sequence(TE, TR; ...)`: fast gradient-echo reference with flip angle α. It has no 180° refocus, so it is T2*-weighted; the implementation includes gradient spoiling only, not full RF-spoiled FLASH phase cycling.

Analytical signal models

- `generalized_ir_signal`: non-spatial IR signal for arbitrary inversion angle. π gives inversion recovery and $\pi/2$ gives saturation recovery.
- `mse_signal`: analytical CPMG echo train, $e^{-k\,\mathrm{ESP}/T_2}$. Used as a fast T2 oracle in tests.

Fitters and forward models

- `fit_t2_se(TEs, mags)`: log-linear estimator for $|S| = S_0 e^{-\mathrm{TE}/T_2}$.
- `fit_t1_ir(TIs, mags)`: conventional long-TR magnitude IR estimator, fitting $|A - Be^{-\mathrm{TI}/T_1}|$.
- `fit_t1_generalized_ir`: main RL estimator. It accepts per-block inversion angles, optional `TRs`, excitation angles and `Npe`; `Npe = nothing` selects the steady-state model and integer `Npe` selects the transient mean over the acquired phase-encode shots.
- `fit_t1_t2_generalized_ir`: joint T1/T2 grid fit for IR-TSE-style data. T2 is identifiable only when TE varies, because a constant TE factor can be absorbed into the fitted amplitude.
- `steady_state_mz_at_excite`, `transient_mz_at_excite_npe` and `transient_mz_per_shot`: shared longitudinal forward models used by the fitters, tests and CR-style baselines.

All image-producing sequences use the same reconstruction path: `raw_to_kspace(raw, Npe, Nfe)` followed by `kspace_to_image(ksp)`. Sphere-centre ROI extraction uses the same coordinate convention through `sphere_descriptor_pixels`, avoiding a separate hand-coded image-space mapping in the experiments. The heavier fit algebra is kept out of the main implementation chapter; Appendix A summarises the derivations that matter for interpreting the estimators.

3.7 KomaMRI integration

`MRISystemPhantom.jl` is built entirely on top of `KomaMRI.jl` and returns standard `KomaMRI.Phantom` objects. The whole phantom is assembled using a single operation: the concatenation operator `+` on `Phantom`. Each voxelised sphere becomes a small `Phantom`, the spheres of each T1, T2 and PD plate are concatenated into that plate, the three plates are concatenated into a sphere stack, and finally the water background is concatenated on top. The result is a single array of spins with heterogeneous relaxation values, which is exactly what `KomaMRI.simulate` expects.

Because the output is an ordinary `KomaMRI.Phantom`, existing KomaMRI features work out of the box, such as the interactive plotting tools which are used extensively in the example scripts.

The matching scanner is obtained from `scanner_for_field(cfg)`, which returns a `Scanner` with B0 set to 3.0 T for `:T3` or 1.5 T for `:T15`. Pairing a phantom with a `Scanner` of a different B0 silently produces the wrong relaxation values - a mistake the author made in practice - so this helper exists to make the coupling explicit.

3.8 Library availability

`MRISystemPhantom.jl` is developed as a standalone open-source Julia package, with source code hosted on GitHub and API documentation hosted on GitHub Pages. The package includes a complete test suite (~373 tests) and documentation pages covering phantom construction, sequence design, parameter fitting, and SNR diagnostics.

An independent, public library has several benefits:

1. It provides a complete, tested forward pipeline — phantom construction, sequence generation, simulation, reconstruction, ROI extraction, and parameter fitting — rather than a set of experiment-specific scripts.
2. It makes the methodology reproducible and extensible for other groups working on quantitative MRI system validation or simulation-based sequence optimisation.
3. It provides a reusable digital twin of a widely deployed calibration device, so simulated methods can be checked against known T1/T2 reference values before scanner deployment.
4. It integrates directly with the Julia/KomaMRI ecosystem by returning standard `KomaMRI.Phantom` and `Sequence` objects.
5. Its example scripts provide runnable, educational references for phantom construction, sequence design, parameter fitting, SNR calibration, and the fidelity checks reported here.

3.8.1 Example scripts

The package ships a set of runnable, self-contained demonstration scripts in `examples/`. Each exercises one slice of the public API and writes interactive Plotly HTML into `src/assets/`.

- `plot_phantom.jl`: renders the T1/T2/PD maps of the full phantom, per-plate axial slices, the fiducial grid, a vertical slice through all three contrast plates, and an augmented rotated/translated phantom. The outputs are interactive Plotly HTML files.

- `plot_fidelity_phantoms.jl`: visualises the phantom geometry (water-coarsening and caching) behind the multi-fidelity RL curriculum. The outputs are interactive Plotly HTML files.
- `t1_mapping.jl`: runs a full IR-SE acquisition, reconstructs the image, extracts per-sphere ROIs and fits T1.
- `conventional_baseline.jl`: runs the conventional fixed-sequence baseline (IR/SE sweep plus fit for all 28 T1 & T2 contrast spheres).
- `snr_calibration.jl`: performs the NEMA MS-1 dual-acquisition SNR measurement.
- `compare_water_coarseness.jl` and `hamming_apodisation.jl`: produce the fidelity-vs-cost and apodisation figures (§2.3) along with the numbers quoted there.
- `plot_seqs.jl`: writes one interactive RF/gradient/ADC waveform plot per sequence into `src/assets/sequences/`. Sequences included: IR-SE (with and without crusher & TR-spoiler variant), SE for T2 mapping, IR-TSE echo-train, and spoiled GRE.

Because the output is a standard `KomaMRI.Phantom`/`Sequence`, these scripts rely only on KomaMRI's own interactive plotting, with no bespoke visualisation code in the library.

4

Validating KomaMRI

KomaMRI is the simulation environment that all RL agents in this project interact with, so simulator validation is a prerequisite for every result in this project. Although KomaMRI is a peer-reviewed, open-source, and actively maintained MRI simulation framework [10], the validation experiments in this project found two floating-point time-discretisation failures. Both were triggered by long cumulative sequence time. The failures were reduced to minimal reproducers and fixes were contributed upstream: issue #779 / PR #780, now merged, and issue #788 / PR #789, open at the time of writing.

This work is part of A1, the validated digital twin and simulator pipeline, and directly addresses C1: simulator validation for qMRI. For A1, the key contribution is the validation of the simulation pipeline rather than the upstream reports alone. The two failures were reduced to minimal reproducers, traced to specific floating-point timing mechanisms, fixed upstream, and then verified by recovering the phantom's known T_1 values in the full qMRI pipeline.

4.1 KomaMRI simulation model

KomaMRI simulates the magnetization of each spin of a `Phantom` for variable magnetic fields given by a `Sequence` [10]. All `Phantom` objects used in this project were produced by `MRISystemPhantom.jl`. Each `Phantom` is a cloud of spin isochromats: groups of nearby nuclei that are assumed to share a single position (x, y, z) and one set of tissue parameters $(T_1, T_2, T_2^*, \rho, \Delta\omega)$. Here ρ is proton density and $\Delta\omega$ is off-resonance: the offset of that isochromat's Larmor frequency from the nominal scanner frequency. Each isochromat is therefore represented by a magnetisation vector (M_x, M_y, M_z) evolving according to the Bloch equations.

The user-facing interface is:

```
raw = simulate(phantom, sequence, scanner; sim_params)
```

Here `scanner` defines the scanner hardware used to play the sequence. In these experiments it is an idealised hardware model: eddy currents and field inhomogeneities are not modelled. This means RF pulses are treated as exact, although in reality a 180° refocus pulse can vary by a couple of degrees across the excited volume.

A `Sequence` is made up of RF pulses, which rotate the magnetisation vector, as well as gradients, delays, and ADC readouts. Internally, `simulate` represents the continuous sequence on a discretised time grid. Gradient events use a time step Δt, while RF pulses are sampled more finely using Δt_{rf}. These time points are then split into blocks. In excitation blocks, where RF is active, KomaMRI integrates the full magnetisation rotation. In precession blocks, where RF is off, the update reduces to relaxation and phase accrual, which can be computed analytically with exponentials and is cheaper to evaluate. At each ADC sample, the simulator sums the transverse magnetisation over all spins to form the complex received signal sig$[t]$.

The key computational assumption is that spins evolve independently and only interact through the final signal sum. This is a standard MRI simulation approximation and is less restrictive than the hardware idealisations above. It is also what makes KomaMRI practical for this project: spins can be partitioned across CPU threads or GPU kernels, then summed only at the receiver. This parallelism motivated the use of KomaMRI and helps overcome C2, the cost of simulation. KomaMRI also models bulk T_2^* decay rather than microscopic spin-spin interaction. Overall, these are standard MRI simulator assumptions that model the relevant physics to an acceptable error, so learned policies should plausibly generalise to real hardware.

4.2 Initial validation failure

The first validation test was deliberately simple. With no added noise, the test simulated an inversion-recovery sweep across the 14 T_1 spheres and fitted the standard monoexponential recovery model:

$$M_z(\mathrm{TI}) = M_0 \left(1 - 2e^{-\mathrm{TI}/T_1}\right).$$

In a noise-free digital phantom, this should be almost exact. Instead, the fitted T_1 values had a mean absolute percentage error (MAPE) of 39.4%, with some spheres close to 100% error. The recovery points

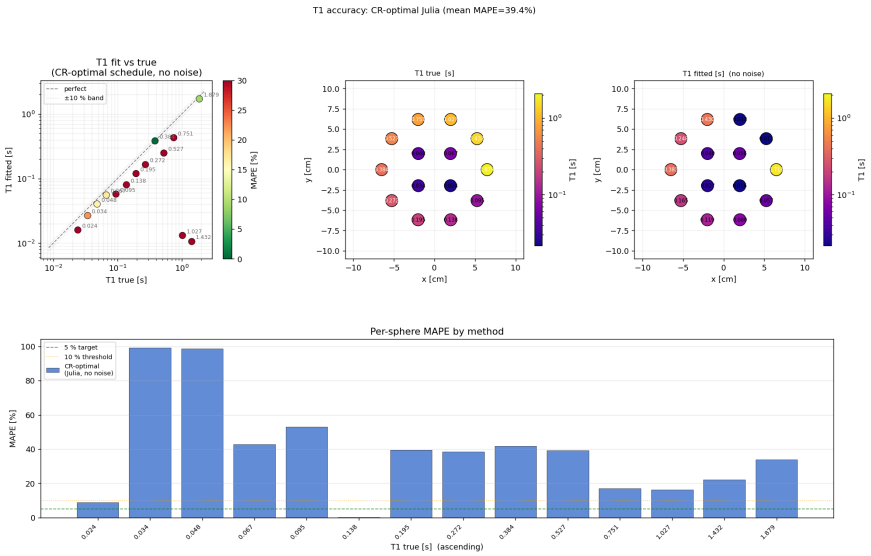


Figure 4.1: Noise-free T_1 recovery before the KomaMRI fixes. The fitted values should lie on the diagonal. Instead the mean MAPE is 39.4%, with only two out of the 14 spheres in the ±10% band.

did not lie on a monoexponential curve. Either the simulator, sequence, reconstruction, or fitting pipeline was wrong, or there was relevant MRI physics not captured by the simple recovery model.

The first step was to rule out plausible MRI explanations. Imperfect transverse spoiling was the most likely candidate: residual M_{xy} can survive one shot, be refocused by later RF pulses, and contaminate the next echo. The standard way to suppress this is to add spoiler gradients, typically around the end of the shot and around refocusing pulses, to dephase any remaining transverse magnetisation. However, when implemented this did not remove the error; it changed mean MAPE from 39.4% to 44.0%. Increasing TR also made the fit worse, even though longer recovery time should reduce both residual transverse coherence and incomplete longitudinal recovery. This was the first strong clue that the failure was driven by total simulated time, not by the physical state at the start of each shot.

Reconstruction explanations were also tested. Hamming-window filtering and ROI masking did not remove the error, making ordinary Gibbs ringing unlikely. A phase-sensitive reconstruction was also checked to ensure the magnitude image was not hiding a sign or phase convention error. These tests changed details of the images, but not the failure mode.

The decisive diagnostic was to inspect k-space directly. Rather than showing random fitting error, the measured k-space collapsed after a fixed amount of simulated time, even though the acquisition should have remained symmetric. This shifted the investigation away from missing MRI physics and toward the simulator's handling of long absolute times.

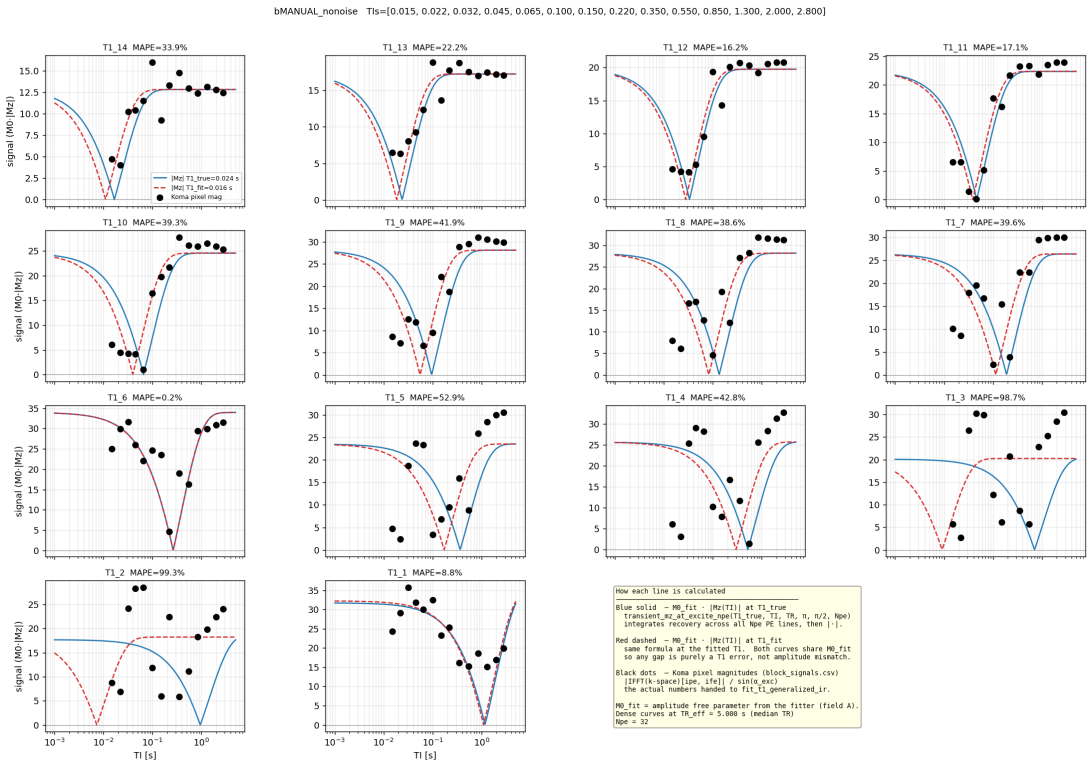


Figure 4.2: Recovery points before the KomaMRI fixes. The simulated samples do not lie on the best-fit monoexponential curves. The deviation is structured, not random, so it points to a repeatable simulation or analysis error.

4.3 RF edge marker collapse

The first minimal reproducer was `koma_bug_minimal.jl`. It removed the phantom geometry and reconstruction and repeated the same simple shot at increasing absolute simulation times. The expected result was a constant per-shot signal. Instead, sharp jumps appeared once cumulative simulated time reached roughly 70–150 s. For example, with TR = 15 s the signal jumped at shot 6 (~90 s) by 21%; with TR = 8 s it jumped at shot 9 (~72 s) by 19%; and with TR = 30 s it jumped at shot 4 (~120 s) by 21%. This showed that the threshold was time-driven, not shot-count-driven, and raised the next question: was KomaMRI discretising the same RF pulse differently at large absolute time?

The RF-edge investigation revealed that the first failure was caused by the way KomaMRI represents sharp RF edges on its simulation time grid. Tracing the RF pulse discretisation led to `points_from_key_times`, and then to the edge-handling logic inside `get_variable_times`. In `get_variable_times`, KomaMRI creates a small set of key times around each RF pulse:

```
points_from_key_times(sort([t1, t1 + MIN_RISE_TIME, tc,
                            t2 - MIN_RISE_TIME, t2]); dt=Δt_rf)
```

where `t1` and `t2` are the start and end of the pulse, `tc` is the centre. KomaMRI encodes the

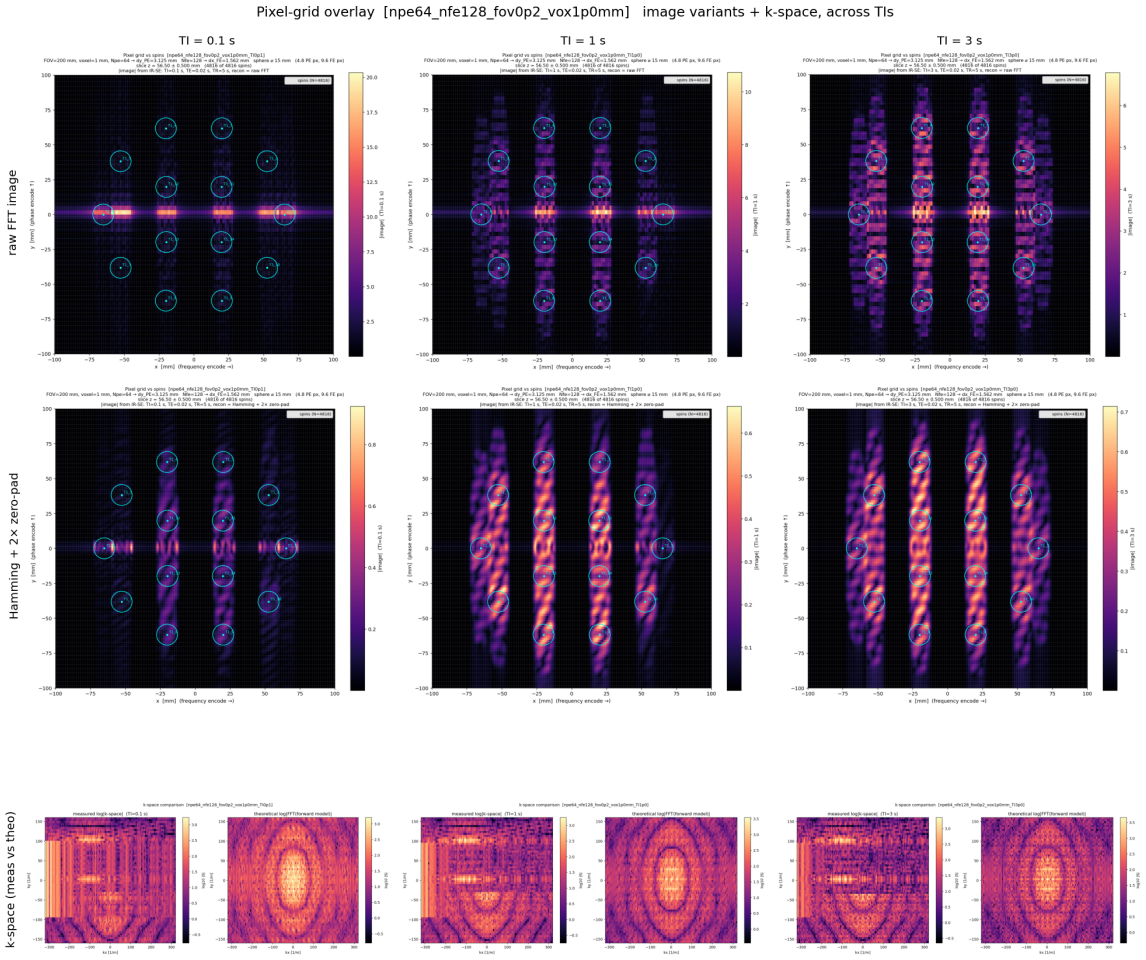


Figure 4.3: Image-space symptom of the timing failure. The figure shows an IR-SE acquisition at three inversion times (TI = 0.1, 1.0, and 3.0 s). The first row shows the reconstructed image, the second row applies a Hamming window, and the bottom row compares theoretical and measured k-space. Because the phase-encode direction (k_y) is acquired shot by shot, moving along k_y corresponds to advancing in simulated time. The early phase-encode lines reconstruct correctly, but later lines collapse into horizontal streaks after a fixed cumulative time. This is too large and structured to be ordinary ringing, and is not removed by the Hamming window. Generated by `pixel_grid_overlay.jl`.

sharp RF edge using a fixed offset $\epsilon = 10^{-14}$ s, implemented as `MIN_RISE_TIME`. The pair `(t1, t1 + MIN_RISE_TIME)` is used to encode the rising edge of the RF pulse. At `t1`, interpolation should return zero RF amplitude; immediately after the edge, at `t1`+ϵ, it should return the full RF amplitude. This creates a near-discontinuous step while still letting KomaMRI store the RF waveform using a small number of key points, with interpolation filling in the values between them.

The problem is that ϵ is a fixed absolute time, but Float64 precision is relative. The spacing between t and the next representable Float64 number, calculated by `eps(t)` in Julia, grows with the magnitude of t: $\mathrm{eps}(t) \approx t \cdot 2^{-52}$. Once $\mathrm{eps}(t)/2 > \epsilon$, adding ϵ no longer produces a distinct Float64 value. In practice this happens at about $t \geq 128$ s. Beyond this point, `t1 + MIN_RISE_TIME == t1` bit-for-bit. The two key times that should define the rising edge therefore collapse onto the same value. When the time vector is sorted and deduplicated, one of them is removed.

The isolated test `koma_bug_isolate.jl` backed this up by asking whether the same 1 ms RF pulse is converted into the same time points at different absolute times:

t_0 [s]	`nraw`	`nunique`	eps(t_0)
0.0	24	23	5.0×10^{-324}
3.0	24	23	4.44×10^{-16}
67.0	24	23	1.42×10^{-14}
99.0	24	23	1.42×10^{-14}
200.0	26	21	2.84×10^{-14}
1000.0	26	21	1.14×10^{-13}

The table shows the collapse directly: early pulses retain 23 unique time points, but at $t_0 = 200$ s this falls to 21 as $\text{eps}(t_0)/2$ exceeds ϵ. This confirms that the RF edge markers have collapsed before simulation.

The collapsed points are not redundant: they represent the two sides of the RF step. At `t1`, RF is zero; at `t1`+ϵ, RF has reached the pulse amplitude. Deduplicating them removes the edge, so the Bloch integrator applies the wrong RF value for one full Δt_{rf} sample. For KomaMRI's default $\Delta t_{\text{rf}} = 5 \times 10^{-5}$ s and $T_{\text{RF}} = 1$ ms, one missing sample is 5% of the RF area. The ϵ-scale timing collapse therefore becomes a 5% flip-angle error, producing a 20–25% signal jump and later a biased T_1 fit.

The fix in PR #780 made the edge markers adaptive:

```
next_time(t) = max(t + MIN_RISE_TIME, nextfloat(t))
prev_time(t) = min(t - MIN_RISE_TIME, prevfloat(t))
```

`nextfloat` and `prevfloat` step to the adjacent representable Float64 value, so the marker remains distinct even at large absolute time. At small times the original spacing is preserved; at large times the marker becomes one floating point step away rather than collapsing. This fixed the first failure and was merged into KomaMRI.

4.4 Closing knot collapse after time rebasing

After PR #780 the direct checks all passed: the isolated RF-edge script was stable to 1000 s, the minimal multi-shot reproducer no longer drifted over the tested shots, and the new RF-area regression test showed that the discretised pulse area matched the expected trapezium area at large absolute time. The original failure mechanism had genuinely been fixed.

However, the full no-noise T_1 validation was still inaccurate. Further investigation revealed another fixed-ϵ collapse, but at a different point in the time pipeline. KomaMRI also separates a block's closing knot, or block edge, using the same fixed offset ϵ in block-relative time. This is initially safe because the local block times are small (e.g. 1 ms RF pulses). Later, however, `get_variable_times` and `get_samples` rebase these local times to absolute sequence time using `T0 .+ t`, where `T0` is the absolute start time of

the block. This creates the same floating-point problem as before: at large values of `T0`, the ϵ gap is below Float64 precision. The closing knot rounds onto the block-end knot, the two times become bit-identical, and a later `unique!` step removes one of them. The marker therefore disappears much later in the code than where it was first created, which made this second failure harder to track down.

The reproducer showed that the remaining timing error still mattered. It used one spin and a gradient-free repeated inversion-recovery shot:

```
180 deg inversion -> TI -> 90 deg excitation -> one ADC sample -> TR delay
```

Every shot is physically identical. At TR = 15 s, all 24 shots should return the same signal. Instead, the first 17 shots were stable, shot 18 jumped by 52%, and later shots settled to a new wrong plateau:

```
shot  1..17  (sim_time <= 255 s): |signal| = 0.476267 stable
shot 18      (sim_time  = 270 s): |signal| = 0.722479 +52%
shot 19..24  (sim_time >= 285 s): |signal| = 0.677159 new wrong plateau
```

The fix in PR #789 re-separates the closing knot after rebasing, when the correct absolute-time spacing is known. In implementation terms, the closing knot is moved to at most $t_{\text{end}} - \max(\epsilon, \text{eps}(t_{\text{end}}))$. The gap is adaptive: it remains ϵ at small times, but widens when Float64 spacing becomes larger. The re-separation is also idempotent: if the closing knot is already distinct, it is left alone. The fix is applied at each rebasing site: both RF sampling paths in `get_samples`, and the gradient timing path in `get_variable_times`. The latter also re-applies an adaptive `_strictly_increasing_knots!` check so gradient knots remain ordered after rebasing. With this patch, the 24-shot reproducer is flat and the KomaMRI tests still pass.

At the time of writing, issue #779 is merged upstream via PR #780. Issue #788 is fixed in a project fork pinned by this repository, with PR #789 still open.

4.5 Validation after the fixes

With both fixes applied, the original no-noise phantom test behaves as expected. The reconstructed images no longer develop time-driven streaks, the recovery points lie on monoexponential curves, and the fitted T_1 values return to the diagonal. Mean MAPE falls from 39.4% to 0.48%, with a maximum sphere error of 1.21%.

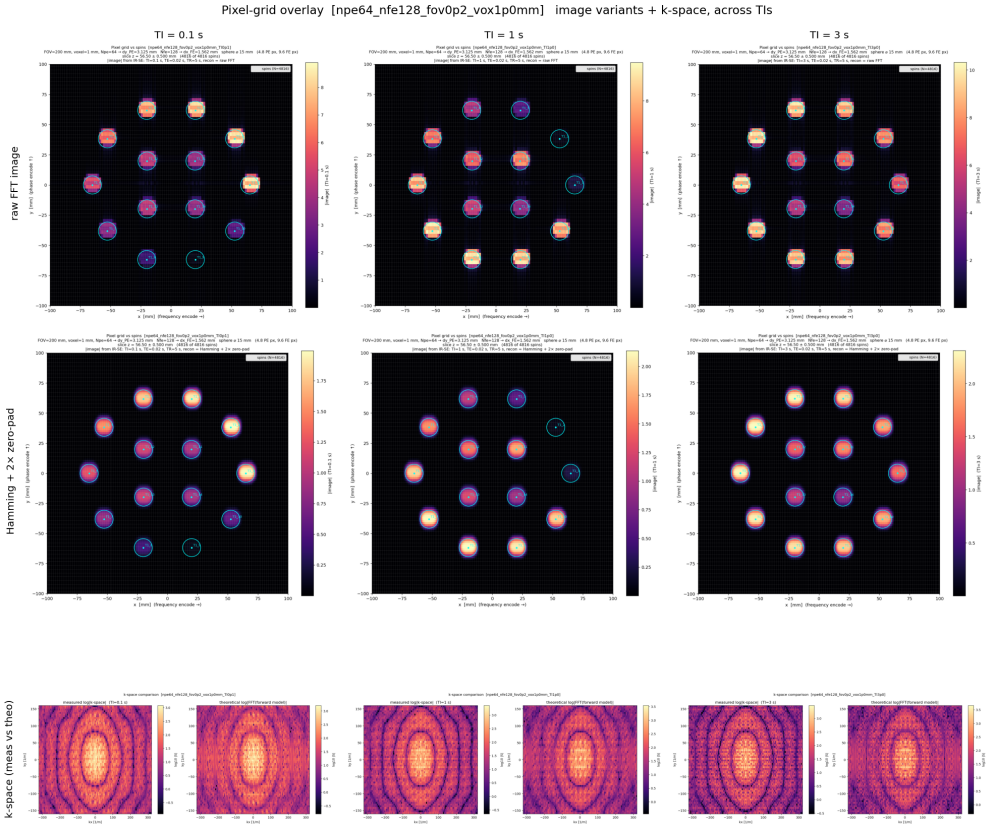


Figure 4.4: Reconstructed images after both fixes. The spheres remain well-localised across the full phase-encode direction. The time-dependent streaking in Figure 4.3 is removed.

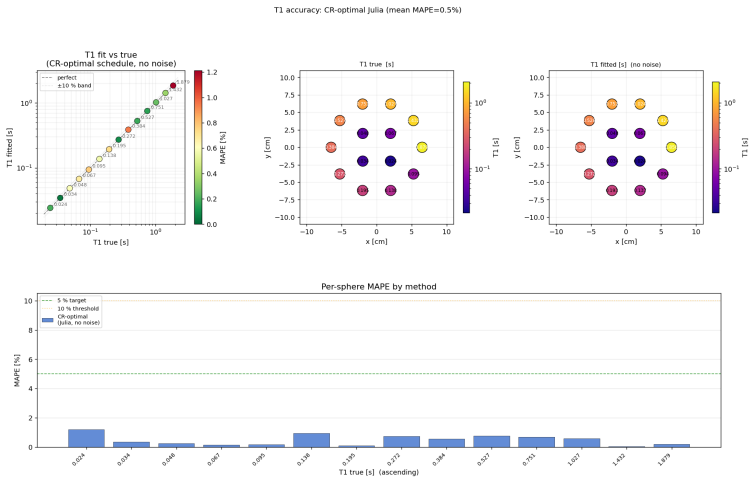


Figure 4.5: Noise-free T_1 recovery after both fixes. Mean MAPE decreases from 39.4% to 0.48%, and all spheres are within 1.21% of their true value. This is the expected accuracy for the noise-free validation case.

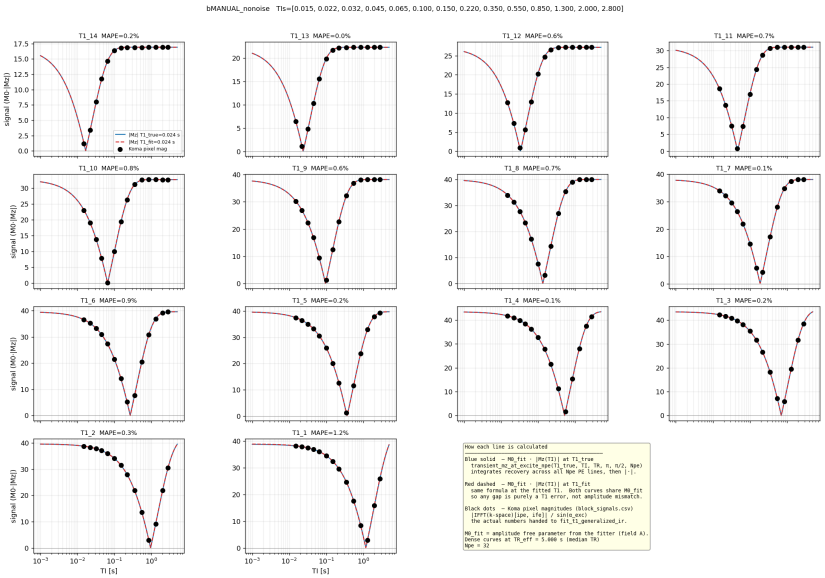


Figure 4.6: Recovery curves after both fixes. The simulated samples now lie on the fitted inversion-recovery curves.

4.6 Evaluation of the fixes

The fixes were evaluated at multiple levels, from isolated timing behaviour to the full qMRI validation task.

Level	Test	Before fix	After fix
Timing representation	RF-edge discretisation at large absolute time	RF edge markers collapse once cumulative time reaches the 70–150 s regime	Stable to 1000 s
Minimal signal reproducer	Identical IR shots, no gradients (`koma_bug_minimal.jl`)	19–21% signal jump once cumulative time reaches 72–120 s	Flat over tested shots
Residual timing reproducer	24 identical IR shots, no gradients (`koma_bug_residual.jl`)	+52% jump at shot 18 (270 s), then wrong plateau	Flat over all 24 shots
Full qMRI validation	No-noise 14-sphere T_1 recovery	39.4% mean MAPE	0.48% mean MAPE (1.2% max MAPE)
Regression safety	KomaMRI test suite	New timing tests fail on unfixed code	Passes after both fixes

Table 4.1: Summary of KomaMRI timing-fix validation.

The minimal reproducers show that the specific floating-point failures were removed, while the phantom validation shows that the corrected simulator produces accurate quantitative maps.

4.6.1 Why this mattered for qMRI

The timing failure had likely gone unnoticed because it is triggered by long cumulative sequence time, while most examples and tests are much shorter, never reaching the ~ 128 s absolute-time regime where Float64 spacing becomes comparable to the fixed ϵ marker. This threshold is still well within the duration of many clinical MRI acquisitions, which often last several minutes. The largest effect also appears for hard RF pulses with sharp edges: losing one Δt_{rf} sample is a large fraction of a 1 ms rectangular pulse. In practical slice-selective sequences, shaped RF pulses and gradient ramps make the same timing error a smaller perturbation.

A second reason is that most MRI simulation checks are qualitative: a reconstructed image can still look plausible when noise and contrast variation hide small systematic signal errors. qMRI is stricter. The validation compares fitted T_1 values against known phantom ground truth, so a systematic signal error directly appears as parameter bias. This project exposed the failure because RL training needs long sequences and qMRI validation needs quantitative agreement, not just plausible images.

This validation covers the IR-SE sequence family used for the main experiments; other sequence families would require the same timing and no-noise recovery checks before being used for quantitative claims.

4.6.2 Why spoiling was a convincing false lead

Although presented here as a single validation narrative, the investigation happened in two stages. Before either fix, transverse spoiling looked plausible because it reduced the catastrophic 924.5% MAPE failure to 92.6%, suggesting that residual transverse magnetisation might be part of the problem. After the first timing failure was fixed, the remaining error was smaller, around 30–40%, and therefore more consistent with a possible sequence effect. Spoiling had to be rechecked before the second timing bug was isolated. In the intermediate simulator shown in Figures 4.1 to 4.3, however, spoiling increased mean MAPE from 39.4% to 44.0%. After both fixes, spoiled and unspoiled no-noise fits are both sub-percent (0.43% and 0.48% mean MAPE respectively). Spoiling remains important for realistic sequence design, but it was not the cause of this validation failure.

5

Adaptive qMRI Sequence Design with Reinforcement Learning

Contents

5.1 Chapter aim

The previous chapters established the simulator and digital phantom. This chapter uses them for the central adaptive acquisition experiment: can a reinforcement learning (RL) agent choose inversion-recovery spin-echo blocks that estimate T_1 more efficiently than a fixed protocol?

This chapter addresses C2 and C3 from Chapter 1. C2 is the compute problem: a full KomaMRI Bloch simulation is too slow to use naively inside RL. C3 is the adaptive design problem: the best next

acquisition should depend on what has already been measured, not only on a fixed protocol chosen before the scan. The main technical contribution is therefore a multi-fidelity RL training loop, plus a controlled evaluation of when adaptivity actually helps.

The conclusion is deliberately nuanced. On the full 14-sphere T_1 plate, the agent learned a non-collapsed adaptive policy, but strong fixed schedules still beat it. On a smaller five-sphere continuous-T_1 task, where the episode really changes from scan to scan, the RL policy does beat the matched fixed log-grid comparator. This is the positive adaptive result, but it should not be overstated as a solution to the full 14-sphere mapping problem.

5.2 Position relative to adaptive MRI work

This chapter sits between three related literatures: adaptive model-based MRI, automated sequence design, and reinforcement-learning control of MRI scanners. It is not the first work to adapt an MR acquisition, nor the first work to use machine learning for sequence design. The narrower contribution is a simulation-in-the-loop RL environment for qMRI in which the agent controls a spatially encoded 2-D acquisition of a calibrated phantom, receives fitted tissue-parameter estimates during the scan, and is evaluated against fixed qMRI protocols.

Work	What it demonstrates	Critical comparison
Adaptive model-based MR [18]	Bayesian update of tissue-parameter uncertainty during a real-time multi-echo experiment, with simulation, phantom and healthy-volunteer validation	Stronger external validation than this project, including real hardware. It is not an RL controller, and the reported task is a model-based adaptive T2/MRS experiment rather than RL control over a 2-D qMRI phantom acquisition.
MRzero [11]	End-to-end differentiable optimisation of sequence parameters, including a clinical 3T scanner demonstration	Much stronger hardware evidence. However, the optimisation is supervised and offline: it learns a sequence, not a closed-loop policy that changes actions from the current fitted tissue estimates.
AUTOSEQ [17]	Bayesian RL-style search over sequence actions in preliminary 1-D physics simulation experiments	Important early evidence that MR sequence design can be framed as sequential control, but the task is deliberately simplified. It does not evaluate adaptive qMRI estimation on a spatially encoded calibration phantom.
Walker-Samuel [26]	DDPG control of a simulated scanner for task-specific active sensing and shape classification	The closest direct deep-RL comparator. It shows an RL agent can actively choose scanner actions, but the objective is shape classification, not calibrated parameter mapping. Walker-Samuel's task is more open-ended in one important respect: the agent has to discover useful scanner behaviour from lower-level controls. This project gives the agent higher-level IR-SE building blocks and a fitter. The trade-off is realism of the quantitative evaluation: this chapter uses 2-D k-space reconstruction, a digital QalibreMD phantom, noise, and fitted T_1 error as the reward. It is still weaker because it remains simulator-only.
MRF optimisation [12], [13]	Offline design of time-varying schedules for multi-parameter fingerprints	Directly relevant to quantitative imaging and multi-parameter contrast, but the schedule is fixed before acquisition. The missing ingredient, which this chapter tests in a narrower T1 setting, is observation-conditioned adaptation during the scan.

Table 5.1: Position of this chapter relative to the main adaptive and automated MRI sequence-design papers discussed in Chapter 2. The comparison is not a claim that this work is broadly superior: existing adaptive MR and MRzero have stronger hardware validation, while this chapter contributes a more realistic deep-RL qMRI control setting than the closest prior RL scanner-control work.

The key distinction is therefore methodological rather than absolute performance. Compared with

the strongest hardware-validated papers, this project is less mature because every result is still simulated. Compared with the closest RL scanner-control work, it is more structured but more quantitative: the agent is not discovering arbitrary pulse-sequence behaviour from scratch, because the sequence family and fitter are supplied. Instead, it must improve fitted relaxation estimates after a 2-D k-space acquisition of known phantom materials. The positive Run B result should be read in that context: it is evidence that RL can learn a useful closed-loop qMRI timing strategy in a more realistic evaluation pipeline, not proof that it is already ready for scanner use.

5.3 Environment formulation

Each episode simulates a 2-D inversion-recovery spin-echo acquisition through the T1 plate of the digital phantom. At each decision point the agent chooses the timing of the next block. The reported runs use $N_{\mathrm{pe}} = 32$, $N_{\mathrm{fe}} = 64$, 1 mm sphere voxels, full background water at target fidelity, and fixed echo time TE = 20 ms. The scan-time budget is 240 s for the main experiments in this chapter. Since one phase-encode line is acquired per shot, each block costs approximately $N_{\mathrm{pe}} \cdot \mathrm{TR}$ seconds.

Noise model. Independent complex Gaussian noise with absolute standard deviation $\sigma = 50$ is added to every k-space sample. The value is calibrated, not arbitrary: the signal scale is a sum over simulated spins, so the $\sigma \rightarrow$ SNR map depends on voxel size, slice geometry and matrix size, and must be derived at the training configuration. At this geometry $\sigma = 50$ gives a NEMA MS-1 dual-acquisition SNR of ≈17 averaged across the T1 plate at the reference block, rising to ≈28 on the brightest spheres (measured by `scripts/e2_snr_report.jl`). This sits squarely in reported clinical inversion-recovery image quality: a normal-brain FLAIR study measured tissue SNRs of roughly 20–30 across the structures analysed [28], while broader fMRI SNR work emphasises the dependence of SNR on voxel volume and acquisition design [29]. The agent is therefore stressed with clinically realistic noise rather than an arbitrary level. A 25-seed noise sweep (Appendix A.6) further shows that T1 fits degrade gracefully up to $\sigma \approx 85$ and collapse beyond, so $\sigma = 50$ leaves the task noise-limited but not estimation-broken.

Action space. The policy outputs a vector in $[-1, 1]^n$ which the wrapper maps to physical timings. The bounds were fixed from the phantom and pulse physics rather than tuned:

- TI $\in [0.01, 3.0]$ s, chosen to bracket the plate's inversion nulls (where signal contrast and T_1 information peak), from the shortest sphere's ≈17 ms null ($T_1 \ln 2$ at $T_1 = 24$ ms) to the longest's ≈1.3 s ($T_1 = 1.88$ s). By 3 s the longitudinal magnetisation has nearly fully recovered for every sphere,

so the signal is insensitive to T_1. The 10 ms floor is not a hard physical limit (RF pulses occupy <1 ms at $B_1 = 20\,\mu$T) but is realistic and never binds.

- TE $\in [5, 80]$ ms, fixed at 20 ms in all reported runs: TE carries T_2, not T_1, information, and freezing it removes a degree of freedom that only dilutes exploration on this task.
- TR $\in [0.5, 5.0]$ s. The upper bound does not reach full recovery for the longest sphere (the $5\,T_1$ rule of thumb would want ≈9.4 s at $T_1 = 1.88$ s): the intended regime is steady-state partial recovery, never a fully relaxed start. The fitter models that partial recovery (the generalised IR model of Chapter 3), so short-TR blocks are genuinely informative rather than merely corrupted, and the agent trades recovery against the $N_{\text{pe}} \cdot \text{TR}$ block cost.
- $\alpha \in [5°, 90°]$ when learned (Run A), spanning the full Ernst regime.

TI uses a logarithmic action mapping, $\text{TI}(u) = \text{TI}_{\min}(\text{TI}_{\max}/\text{TI}_{\min})^u$ for $u \in [0, 1]$, giving constant action resolution per decade of T_1; under a linear map the informative TIs for the shortest spheres occupy roughly the bottom 1.5% of the interval, which starves them of policy resolution. In the later five-sphere ablations α was fixed at 90°. This was an evidence-based simplification: in Run A the learned flip angles collapsed to 90°, and the Cramer–Rao fixed-schedule baseline independently chose 90° because it reduced the CRLB. The flip-angle degree of freedom was therefore redundant for these T1 experiments, and later runs spent the capacity on timing decisions rather than re-learning the same boundary solution.

Action repair. The environment repairs infeasible timing requests rather than terminating the episode: the executed repetition time is

$$\text{TR}_{\text{exec}} = \max(\text{TR}_{\text{req}},\ 0.5,\ (\text{TI} + \text{TE})/0.9)\,,$$

i.e. TR must contain the inversion and echo with 10% headroom. Hard rejection would be cleaner semantically, but early random policies would burn rollouts learning the action-space geometry instead of sequence design, and the resulting penalties push PPO towards conservative long-TR choices. The guardrail is audited rather than hidden: every step emits both requested and executed timings plus a repaired flag, and the evaluation scripts report repair rates and mean TR lift. A policy that leans on the repair is therefore detectable; the reported Run B 240 s policy executes zero repairs, and the 560 s policies repair 0.3–1.8% of blocks.

Observation. The observation is deliberately compact: $\log_{10}$ of the running per-sphere T_1 estimates (a zero sentinel before the first valid fit, which requires at least two measurements), the fraction of scan time used, the fraction of blocks used, and a constant bias input. Observations and rewards are

normalised online with clipping at ±10. The agent never sees the reconstructed image or raw k-space: it acts only on the fitter's T_1 estimates and budget, the same interface a qMRI pipeline would expose. This keeps the policy on the genuine estimation objective rather than on simulator-specific pixel patterns that may not generalise to real hardware. Some ablations append an uncertainty proxy from the fitter (Section 5.7); the main 240 s result does not. After each block, the environment reconstructs the image, extracts each sphere's region-of-interest signal, refits T_1, and computes the reward. The main reward is the improvement in log-MAPE:

$$r_t = \log(\mathrm{MAPE}_{t-1} + \epsilon) - \log(\mathrm{MAPE}_t + \epsilon)\,.$$

This reward is dense and local: it rewards a block for reducing the current estimation error, rather than only awarding a terminal score. This was important because an initial single-voxel experiment (not reported here) collapsed to a nearly fixed policy when a large terminal bonus made it sufficient to finish the episode rather than optimise the measurement sequence.

Training uses PPO [24]. PPO was chosen because it is a robust on-policy baseline and can be used with continuous actions without needing a replay buffer. This matters for the Python–Julia simulator loop: each rollout step triggers simulator, reconstruction, fitting, and reward code across the Python–Julia boundary. During development, a simple on-policy learner made it easier to diagnose whether failures came from the MRI model, the fitter, the reward, or the policy update itself.

5.4 Multi-fidelity curriculum

5.4.1 The cost wall and the fidelity ladder

The main engineering difficulty is simulator cost. Every environment step triggers a multi-shot Bloch simulation of the whole phantom, followed by 2-D reconstruction and a refit. The cost is set mainly by how many RF pulses the integrator must resolve: KomaMRI steps through the free-precession delays (TI, TR) with large time steps, but each RF pulse needs fine time discretisation, so the per-step cost scales with the number of shots N_{pe} (one excitation and refocusing pulse per phase-encode line) and the spin count n_{spins}, and only weakly with the requested timing. At the target configuration (T15, 64×32, 1 mm voxels, 1 mm slab through the T1 plate) the sliced phantom contains 23,735 spins, of which 18,919 (80%) are background water. The sphere material that the agent actually measures is only 4,816 spins. A full-Bloch step measured about 4 s on CPU, so a 200k-step PPO run is roughly nine days of simulation before any evaluation overhead. The reported Run B family was trained on GPU, where the same stages

ran some 5–7× faster (≈0.06 s/step at the cached and coarse-water stages); extrapolated by spin count to the 1 mm full plate this is still ≈0.2 s/step, leaving a single-fidelity full-Bloch run on the order of a day before evaluation overhead. Single-fidelity full-Bloch training therefore does not fit the project compute budget on either device.

Because 80% of the spins are water, both cost levers target the water: *coarsen* it or *cache* it. Coarsening reuses the digital twin's independent water grid (Chapter 3): water spins are laid out on a coarser grid with proton density rescaled to conserve total magnetisation, and a 3 mm water grid cuts the water spin count by 9× (18,919 → 2,103), taking the phantom to 6,919 spins. Caching, described next, removes the water from the per-step simulation entirely. Together with the analytic surrogate these give the ladder in Table 5.2, spanning a ∼30× per-step cost range.

Fidelity	Water model	Bloch spins	s/step	Main bias
`analytic`	closed form	0	0.034	no reconstruction, no water crosstalk
`cached3`	cached template, 3 mm	4,816	0.34	cached and coarse water
`cached`	cached template, 1 mm	4,816	≈0.34	cached water, target geometry
`full3`	full Bloch, 3 mm	6,919	0.42	coarse water discretisation
`full`	full Bloch, 1 mm	23,735	1.03	target fidelity

Table 5.2: Fidelity ladder used for E2. "Bloch spins" is the number simulated per step at the 14-sphere plate (exact counts from `scripts/e2_spin_counts.jl`; the cached rungs Bloch-simulate only the 4,816 sphere spins and add the water from the template); s/step are measured Run A CPU stage averages, set mainly by N_{pe} and the spin count rather than the requested timing. The `cached` rung costs the same as `cached3` because the template add is resolution-independent; its standalone benchmark measured 8.0× versus matched full Bloch. Lower rungs are cheaper but biased; Section 5.4.3 ensures the bias cannot validate itself.

5.4.2 Cached water: exploiting simulator linearity

The cached rungs rest on two exact structural facts. First, the Bloch simulation is linear in spins, so the acquired k-space separates exactly:

$$S_{\text{full}} = S_{\text{spheres}} + S_{\text{water}}.$$

Second, the water is a single homogeneous material, so its contribution to phase-encode line k (acquired on shot k) factorises into a scalar shot amplitude and a fixed geometric profile:

$$S_{\text{water}}[k,:] = M_z^{(k)}(\text{TI}, \text{TR}, \alpha)\, \sin\alpha\; W_\alpha[k,:].$$

Here $M_z^{(k)}$ is the water's longitudinal magnetisation at the k-th excitation, available in closed form from the same transient recovery model the fitter inverts, and W_α is a template capturing everything geometric — spin positions and Fourier encoding, T_2 decay at the reference echo time, static B_0 phase. The template is obtained once, by running a single Bloch simulation of the water alone and dividing the $M_z^{(k)} \sin\alpha$ factor out of each k-line.

Evaluating a new (TI, TR) then costs $O(N_{\mathrm{pe}})$ arithmetic for the new $M_z^{(k)}$ plus a row-wise rescale of the template; only the sphere spins are re-simulated. This is the $\sim 8\times$ per-step saving of the cached rungs. The alternative of modelling the water analytically as well is measurably too crude: it leaves visible reconstruction and crosstalk residuals and per-sphere T_1 deviations from the full simulation of up to $\sim 40\%$, whereas the cached model matches full-Bloch T_1 fits to 0.12% on average (worst sphere 1.15%, the fitter's discrete-grid floor). Appendix A.7 shows the validation images and fits.

The flip angle is the subtle part: the factorisation is exact only at the α the template was built with. Two effects were measured. First, the closed-form transient mis-scales across α: at 90°, $\cos\alpha = 0$ and every shot starts from the same thermal state, but at 30° residual magnetisation carries between shots and the ratio of simulated to analytic per-shot amplitude drifts from 1.01 on the first k-line to 1.40 on the sixteenth. Second, the template itself changes with α through spoiling and echo formation, by 7–21% spatially between a 30° and a 90° template; this error is TI-independent, so no TI rescaling can correct it. The implementation therefore stores a *bank* of α-matched templates on a grid spanning the action range. At a matched α a template is exact to $\sim 10^{-8}$ relative k-space error; an off-grid α is evaluated by computing the two bracketing templates, each at its own α, and blending linearly in α. A template is never rescaled to a different flip angle analytically — that would reintroduce exactly the error the bank exists to avoid.

One constraint follows from the construction: the template is keyed to the spin positions, so the cached rungs hold the phantom pose fixed, while the full-Bloch rungs and all full-fidelity probes apply the in-plane pose jitter. Cheap stages therefore optimise under slightly narrower conditions than the target measures — a stated fidelity compromise, accepted because every promotion and checkpoint decision is scored on the full environment.

5.4.3 The switch rule: bias-aware promotion

Warm-starting between fidelities is mechanically trivial; the design question is *when* to promote. Fixed per-stage step counts are impossible to justify and waste budget. The switch rule adapts two ideas from multi-fidelity RL: Sifaou and Simeone select the fidelity with the best expected information gain about the target policy per unit simulation cost [30], and Cutler, Walsh and How promote when lower-fidelity samples stop being useful for the real task [31]. Neither is implemented literally as this project maintains no posterior over policies, but their shared principle fixes the central design decision: **promotion is driven by held-out full-simulator validation, never by the cheap simulator's own score.** A biased simulator can keep improving its own score by exploiting its bias, the same failure mode as the single-voxel fitter exploit, so a cheap rung may generate rollouts but may not certify its own policy.

Concretely, every fourth PPO rollout (2,048 steps at $n_{\text{steps}} = 512$) the current policy is probed for four episodes on the current-fidelity environment and four on the full environment, with identical seeds. The full-probe score is

$$P_H = -\log_{10}\big(\text{clamp}(\text{MAPE}_H, 10^{-3}, 1)\big)\,,$$

where the clamp models each rung's accuracy floor so a saturated cheap stage reads as flat. After a per-stage minimum-step gate, the trainer promotes out of a cheap stage when any of four signals fires:

1. **Target plateau** (primary): the relative gain in P_H over the last three decision points falls below 2% — the cheap rung has given what it can.
2. **Ranking breakdown**: the Spearman correlation between cheap-probe and full-probe MAPE across the stage's decision points falls below a per-rung guardrail (0.7 for `analytic`, 0.8 for the cached rungs, 0.9 for `full3`), evaluated only once at least four points exist. A cheap rung may be biased, but it must still *rank* policies like the target.
3. **Bias intolerance**: the cheap/full probe gap exceeds one percentage point of mean MAPE or two points of p90.
4. **Budget reserve**: remaining wall-clock falls to the 20% of the total budget reserved for the final full stage. This overrides every other gate, guaranteeing the target fidelity always receives compute.

Each stage also has a hard step cap, and the whole curriculum runs under a hard wall-clock deadline (9 h for the reported runs). On every promotion the *fidelity gap* is recorded — the full-probe MAPE of the warm-started policy before any training at the new rung, minus the previous stage's final value — as a direct transfer diagnostic. The decision logic is a pure Python module, unit-tested on synthetic metric histories separately from the PPO and Julia orchestration, and every decision is appended to a JSON history together with the signal that fired, so each run's switching behaviour can be audited afterwards.

5.4.4 Next-fidelity lookahead (V2)

The plateau rule has a blind spot: it says the current rung is exhausted, not that the next rung will help. The V2 controller adds a cheap empirical test before promoting on progress grounds. A lookahead is triggered when the recent slope of P_H per wall-clock second falls below 0.25× the stage's previous best slope (a scale-free collapse test that transfers across hardware), or when the relative gain comes within 2× of the plateau threshold. The trainer then *clones* the policy into a fresh PPO instance attached to the next-fidelity environment — weights copied, observation-normalisation statistics deep-copied, reward normalisation reset — trains the clone for a single rollout (512 steps, ∼230 s at `full3` in Run A),

and re-scores it on the same full-probe seeds. Promotion on slope grounds happens only if the clone's improvement per second beats the current rung's by a factor of 1.15.

The lookahead carries its own guardrails: its cumulative wall-clock is capped at 10% of the total budget; it never runs into the final-stage reserve; and the clone is discarded afterwards, never mutating the live model or its normalisation statistics — the real model reaches the next rung by warm start, so the one-rollout clone is a probe, not a head start. The plateau rule remains as the fallback, and the logged switch reason distinguishes a lookahead-driven promotion from a fallback one.

5.4.5 Stage hand-off and global-best checkpointing

Only the forward-model and water settings change between rungs; the observation and action layout, reward, noise and resolution are held fixed so the warm-started weights remain valid. The observation-normalisation statistics carry across each switch — the observation is physical ($\log_{10} T_1$ in seconds), so its running statistics are comparable across fidelities — but the reward normalisation is reset, because the reward scale shifts with each rung's bias. Optionally the optimizer is also rebuilt at each switch, so stale Adam moments from the previous rung do not distort the first updates at the new one.

Finally, the trainer maintains a global-best checkpoint across the whole curriculum, selected on the target fidelity. This was added after Run A showed that the final policy of a curriculum can be worse than one it had already passed through (Section 5.5). Selection is two-tier because four-episode probes are far too noisy to pick checkpoints: every full probe (switch decisions, stage starts and ends, periodic stage evaluations) only *screens* candidates, and a candidate that beats the incumbent on its screen is re-evaluated on twelve independent episodes at a separate seed offset. Only a confirmed improvement overwrites the checkpoint, and its metadata records both the screening and the confirmed scores. Report-grade numbers are taken from neither: all tables in this chapter come from fresh 24-episode evaluations with confidence intervals.

5.5 Run A: full 14-sphere T1 plate

Run A was the first end-to-end V2 curriculum. It used the full 14-sphere T1 plate, a 240 s scan budget, $T15$ field values, $\sigma = 50$, fixed TE, and learned α. The ladder was

$$\texttt{analytic} \rightarrow \texttt{cached3} \rightarrow \texttt{full3} \rightarrow \texttt{full}.$$

The run was trained under a 9 h wall-clock budget.

Stage	Steps	Wall [h]	s/step	End full-Bloch MAPE
`analytic`	8.2k	0.08	0.034	16.60%
`cached3`	41.0k	3.28	0.338	**10.37%**
`full3`	73.7k	3.94	0.416	14.03%
`full`	79.5k	1.64	1.03	12.26%

Table 5.3: Run A stage summary. The productive stage was `cached3`; moving to `full3` degraded the target-fidelity score, and the final full stage did not recover the best cached checkpoint.

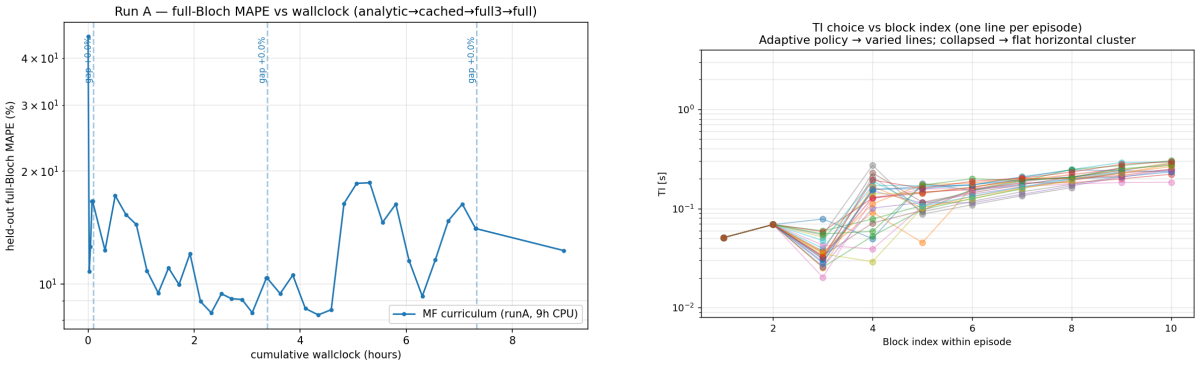


Figure 5.1: Run A curriculum behaviour. Left: full-Bloch probe MAPE versus wall-clock time. The cached-water stage reached the best basin, then full3 regressed. Right: the learned policy was not collapsed; TI varies across episodes and within an episode, but the final policy still under-sampled the long-T1 regime.

Figure 5.1 is the main diagnostic, and the logged switch reasons make the controller's behaviour concrete. The analytic stage was left on a *ranking breakdown* (the cheap probe stopped ordering policies like the full probe) after 8.2k steps; it had already collapsed the full-probe MAPE from ≈42% to ≈10% essentially for free. The `cached3` stage was left on a *target plateau* at 41k steps, having improved the full-probe score into an 8–10% basin — the best the run ever reached. The problem occurred after the switch to `full3`: the target-fidelity score jumped back up and ended worse than the cached basin, and that stage was only left on the *budget reserve* override at 74k steps, handing the last 20% of wall-clock to the `full` stage. The 3 mm-water full-Bloch environment was therefore not a harmless bridge to the 1 mm target; its optimum differs from the 1 mm optimum the probe scores (rank correlation ≈0.6, bias ±5%).

The lookahead mechanism gave a useful warning at exactly this boundary. When the `cached3` slope collapsed, the controller cloned the policy, trained it for one rollout (≈230 s) at `full3`, and re-scored it: the clone's full-Bloch MAPE got *worse* (10.37% → 11.69%), so the slope gain was ≈0 and the rule correctly *declined* to promote on slope grounds. The plateau fallback promoted anyway, and the regression the lookahead had predicted in 230 s then played out over the whole 3.9 h stage. This is an informative negative result: the V2 signal was meaningful, but the controller's action space did not include "skip this rung and keep the global best", which is what the evidence called for.

Run A was evaluated on a strict seed-600000 held-out set. This seed was not used for policy training, curriculum switching, or checkpoint selection, so the negative result in Table 5.4 is not a validation leakage artefact.

Policy or schedule	Mean MAPE	95% CI	p90	Mean time
`cr_optimal_alpha` fixed schedule	**4.70%**	[4.42, 5.00]	**5.68%**	240 s
`log_grid_trmatched` fixed schedule	5.80%	[5.24, 6.40]	8.06%	218 s
Run A cached-best checkpoint	9.44%	[8.18, 10.94]	14.11%	231 s
Run A final policy	11.68%	[10.42, 13.03]	15.22%	232 s
Run A analytic checkpoint	16.03%	[13.97, 18.56]	20.87%	223 s

Table 5.4: Run A comparison on 24 strict seed-600000 held-out evaluation episodes. The agent was adaptive, but the fixed schedules were better on the full 14-sphere task.

The learned policy was nevertheless not the degenerate fixed-policy failure mode. Diagnostics over 24 episodes gave TI intra-episode log standard deviation 0.281, inter-episode log standard deviation 0.290, and modal-bin share 11.9%. The chosen TI also correlated positively with the running estimate and with the most uncertain sphere ($r = +0.50$ and $r = +0.64$ respectively). In other words, the policy was using the state, not replaying one fixed list of actions.

The failure was more specific, and Table 5.4 reads as an unintended ablation of the curriculum itself: *the cheap stages delivered the result and the expensive stages eroded it.* The `analytic` checkpoint scores 16.03%, the `cached3`-best checkpoint 9.44% — the best the run produced — and the final policy only 11.68%. The mechanism is visible in the learned TIs. The cached-best policy had pushed its TI ramp up to ≈0.8 s, long enough to characterise the mid-to-long-T_1 spheres, cutting Pool 2/3/4 ($T_1 = 1.43/1.03/0.75$ s) error to 18.7/13.8/10.7%. The two expensive stages then shrank the ramp back to 0.3–0.6 s and reduced intra-episode exploration (TI log-σ 0.38 → 0.28), re-inflating those same pools to 28.2/23.8/19.0%. The longest sphere, Pool 1 ($T_1 = 1.88$ s), was beyond reach for every checkpoint (≈41%): even the cached-best 0.8 s TIs fall short of its ≈1.3 s inversion null, so this is an action-coverage ceiling, not only a training-time problem.

The flip-angle degree of freedom told a consistent story. It collapsed to the 90° bound and, if anything, the curriculum drove it there harder over training (α std 2.4° → 1.6° → 0.0° across the checkpoints); the best fixed Cramer–Rao baseline also selected 90°. This is why α was fixed in the later Run B family: both the learned policy and the CRLB baseline indicated that the useful optimisation pressure was in TI/TR timing, not flip-angle selection.

One caveat on the absolute numbers. The sibling digital twin was improved mid-project (a random-phantom sampler and an SO3 pose sampler), so the same saved Run A policy scores 11.68% on the current simulator versus 10.16% when first analysed. Table 5.4 re-evaluates every policy and baseline on the *current* simulator, so the comparison is internally consistent; the cross-version shift is reported only to explain why older logs differ.

The Run A interpretation is therefore:

- multi-fidelity training made the experiment computationally feasible;
- the policy learned a genuine state-dependent timing strategy;

- the final 14-sphere agent did not beat fixed schedules;
- global-best checkpointing is required because later fidelity stages can erase a useful cheap-stage policy;
- the full 14-sphere task rewards robust global coverage, so it is not the best test of closed-loop adaptivity.

5.6 Run B: five-sphere continuous-T1 task

Run B changes the task so that adaptivity has a fairer chance to matter. The physical sphere labels are fixed at $\{1, 3, 6, 8, 14\}$, covering the long, mid and short ends of the T1 plate. However, their T_1 values are sampled continuously each episode over the T15 phantom range, with T_2 and T_2^* preserving their nominal ratios to T_1. The geometry is therefore stable, but the realised relaxation distribution changes from scan to scan.

This is easier than the full 14-sphere problem, and it is important to say so. It is not a replacement headline for full phantom mapping. Its purpose is to test the specific adaptive claim: when the episode's tissue parameters vary, can the agent use early measurements to choose a better later timing schedule than a fixed grid?

The main completed Run B result is the 240 s no-uncertainty GPU run. It used the ladder

$$\texttt{analytic} \rightarrow \texttt{cached3} \rightarrow \texttt{cached} \rightarrow \texttt{full3} \rightarrow \texttt{full}.$$

The extra `cached` stage is the target-resolution cached-water bridge missing from Run A. The reporting checkpoint is the global-best policy, not the last policy.

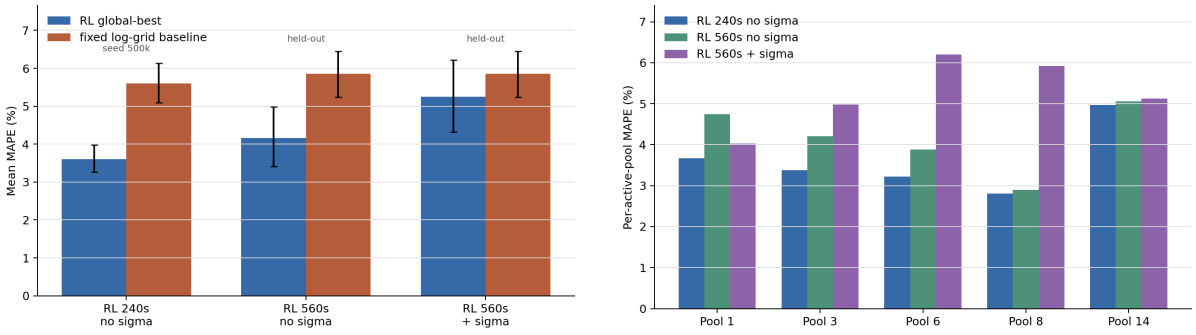


Figure 5.2: Run B five-sphere results. Left: global-best RL policies compared with the matched fixed log-grid baseline. The 240 s result is baseline-comparable but not a strict held-out seed; the 560 s bars use strict held-out evaluation. Right: per-active-pool MAPE for the RL policies.

The 240 s policy beats the matched fixed log-grid comparator by about two percentage points of mean MAPE. The per-pool breakdown is also balanced: Pools 1, 3, 6, 8 and 14 scored 3.67%, 3.38%, 3.22%,

Policy or schedule	Mean MAPE	95% CI	p90	Success < 5%
Run B RL global best	**3.61%**	[3.26, 3.98]	**4.55%**	**91.7%**
Run B RL final policy	4.19%	[3.82, 4.58]	5.26%	83.3%
Fixed `log_grid_trmatched`	5.60%	[5.09, 6.13]	7.08%	–

Table 5.5: Run B 240 s five-sphere result on 24 seed-500000 evaluation episodes. This seed was also used by training evaluation, so the result should be confirmed with a strict held-out seed before becoming the sole headline table. It is nevertheless the matched comparison against the available 240 s fixed baseline.

2.81% and 4.97% MAPE respectively. This matters because the result is not an average dominated by one easy pool. The shortest-T_1 active pool remains the hardest, but it is still below 5% MAPE.

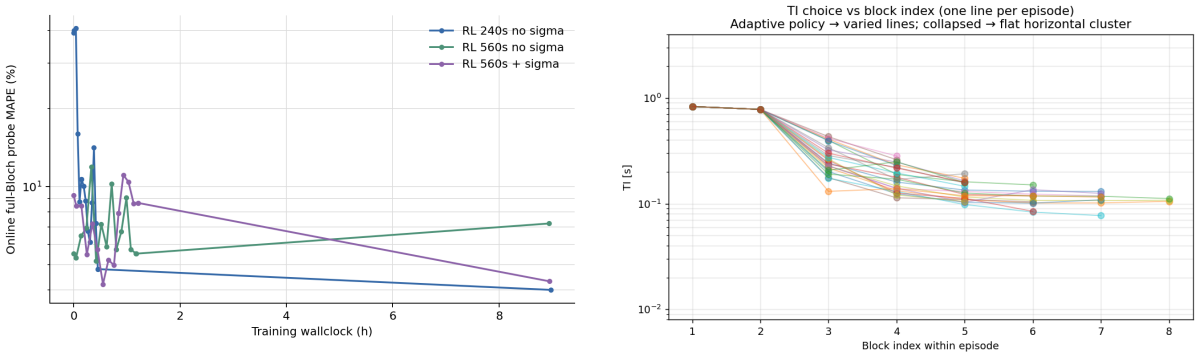


Figure 5.3: Run B 240 s policy behaviour. Left: full-Bloch probes during the curriculum. The final full stage is noisy and global-best selection is needed. Right: the global-best policy opens with a high-TI probe and then uses shorter adaptive refinements.

The learned schedule in Figure 5.3 is qualitatively different from Run A. The global-best 240 s policy uses 5.92 blocks on average, with no timing repairs. It usually starts with two high-TI probes around 0.78–0.83 s and then moves to shorter refinements. The correlation between TI and current T_1 estimate is negative ($r = -0.720$ for the main diagnostic), which initially looks counter-intuitive. It is better interpreted as a two-phase strategy: first buy a long inversion probe that is informative across the sampled range, then shorten TI after the estimate has stabilised. This is a real adaptive pattern, but it is not the simple monotonic rule "higher current T_1 implies longer next TI".

The 240 s comparison shares its evaluation seed with the training stream, so the cleanest statistical claim comes from a longer-budget control trained with α fixed at 90° and evaluated on a strict, never-seen seed-600000 held-out set. At a 560 s budget the global-best RL policy scores **4.16%** MAPE ([3.41, 4.99], p90 6.64%) against **5.86%** ([5.24, 6.45], p90 8.02%) for the matched fixed log grid on the same held-out call — the same ≈1.7 percentage-point advantage as at 240 s, now without any train/eval seed overlap. The fixed log grid is essentially flat from 240 to 560 s (5.60% → 5.71%), so the RL gain is not simply "more blocks": the 560 s policy uses its ≈14 blocks (versus the fixed grid's 10) in a different, closed-loop allocation, and the 240 s policy is the stronger efficiency story, beating the fixed schedule with only ≈6 blocks and ≈230 s.

This is the strongest evidence for A3. The task was deliberately constructed so that a fixed global schedule is less naturally optimal than in the 14-sphere case, and the RL policy then found a compact

closed-loop strategy that improved both mean and p90 error against the fixed comparator at two budgets. The result is still bounded: it rests on one training seed per arm, and the headline 240 s global-best policy still needs a strict seed-600000 re-evaluation to fully remove overlap with the training evaluation stream.

5.7 560 s and ablation status

The longer-budget and observation ablations are not yet mature enough for a full results section, so they are summarised here.

- **560 s no-uncertainty control.** The global-best policy scored 4.16% MAPE on a strict 24-episode held-out evaluation, versus 5.86% for the fixed log-grid comparator on the same held-out call. This supports the five-sphere adaptive result, but it is not a clean scan-time ablation of the 240 s run because the 560 s runs fixed $\alpha = 90°$ while the 240 s run still learned α.
- **560 s uncertainty-channel treatment.** Adding $\log_{10}(\sigma_{T_1}/T_{1,\mathrm{est}})$ to the observation did not improve strict held-out performance. The global-best policy scored 5.25% MAPE with p90 9.34%, worse than the no-uncertainty control. The uncertainty proxy is therefore not a positive result. It may be too noisy early in the episode, when the fitter has only a few measurements.
- **Global best versus final policy.** The 560 s no-uncertainty run had an earlier confirmed global best near 3.45% on the 12-episode confirmation eval, but the final policy drifted to 7.12% on the seed-500000 24-episode eval. This reinforces the Run A lesson: the last policy is not automatically the best policy in a multi-fidelity curriculum.
- **Memory ablations.** Two memory mechanisms are planned but not yet report-grade: an executed-TI histogram in the observation and a recurrent PPO policy with an LSTM state. These test whether the current state $[T_{1,\mathrm{est}}, \mathrm{budget}]$ is an adequate summary of the acquisition history. The hypothesis is concrete: two different TI histories can produce similar current fits, but have different information value for the next block.
- **ROI and reconstruction ablations.** ROI radius 1 was used for Run B to reduce centre-pixel sensitivity. A paired ROI 0/1 evaluation is still needed before attributing gains specifically to adaptivity rather than improved measurement extraction.

5.8 Limitations

The main limitation is scope. The positive result is a proof of concept on a controlled five-sphere task, not a solved full-phantom mapping problem. That task was deliberately engineered so that adaptivity

could matter: the geometry was fixed, the active tissues were few, and the relaxation values changed between episodes. This is the right experiment for isolating closed-loop timing, but it is not yet evidence that the same policy class scales to all 14 T1 spheres, the T2 and PD plates, or pose uncertainty.

The second limitation is task size. With five active spheres, the state estimate is compact and the reward signal is relatively direct. A full plate introduces a different regime: the policy must cover long and short relaxation times at once, avoid sacrificing hard pools for easy pools, and handle a larger set of coupled estimation errors. Run A shows this distinction clearly. It learned a state-dependent policy, but robust fixed coverage remained stronger.

The third limitation is statistical breadth. The final tables use 24-episode evaluations and one training seed. This is adequate for the current conclusion: Run A is a strict held-out negative result, and Run B is a promising controlled positive result. It is not enough to claim that the method is insensitive to PPO initialisation, curriculum timing, or stochastic simulator noise.

The fourth limitation is algorithmic breadth. PPO was a pragmatic baseline, not a claim that on-policy learning is the best optimiser for this problem. Because simulator samples are expensive, an off-policy actor-critic such as SAC could be a better fit: replay would reuse costly transitions and might reduce the number of simulator calls needed to reach a useful policy. That comparison was not run, so the results should be read as evidence that adaptive qMRI can be learned with a robust baseline, not as evidence that PPO is the most sample-efficient or highest-performing RL method for the task.

The fifth limitation is external validation. The original project plan included testing the learned protocol on real scanner hardware, but this was not reached. All results in this chapter are therefore simulator results. They test whether closed-loop sequence design can work inside the KomaMRI-based digital twin; they do not yet show that the learned timings transfer to scanner constraints, sequence programming details, reconstruction pipelines, or real phantom data.

A related geometric simplification is the slice thickness. To keep the spin count tractable, the reported runs image a single 1 mm axial slab through the T1 plate, whereas a real slice-selective acquisition would excite a 3–8 mm slice and so average over more through-plane water and partial-volume material. This makes the simulated scene cleaner than a clinical slice. The choice is not believed to change the qualitative result: experiments with thicker slabs gave similar T_1-fit-versus-truth behaviour, because the fitted quantity depends on the per-sphere recovery curve rather than on the absolute spin count, and the noise level is recalibrated to the geometry in any case (Appendix A.6). A thicker slice mainly raises simulation cost and partial-volume blurring, both of which the multi-fidelity machinery is designed to absorb, but a slice-selective study at clinical thickness remains future work.

Finally, the action space is still narrow. The current policy tunes an IR-SE-style block, mostly through

TI and TR. It cannot choose a different sequence family, change the phase-encode budget, or stop early to trade accuracy against scan time. The chapter should therefore be read as an adaptive T1 timing study, not as a general autonomous MRI protocol designer.

5.9 Future work

The next algorithmic step is to improve the curriculum controller. Run A showed that `full3` can be a biased rung; Run B showed that global-best checkpointing protects against late drift. A simpler ladder,

$$\texttt{analytic} \rightarrow \texttt{cached} \rightarrow \texttt{full},$$

should be tested against the current five-rung ladder. A more ambitious version would sample from several fidelities in parallel rather than treating the ladder as a serial hand-off. In the current runs, the GPU is largely idle during the analytic and cached stages; a parallel design could keep full-Bloch probes running while the cheap simulators generate high-throughput rollouts. Longer term, this becomes pooled multi-fidelity learning: cheap and expensive rollouts are used together with bias correction or control variates. This direction is directly connected to multi-fidelity control-variate and Gaussian-process RL methods [32], [33].

The next state-representation step is to make acquisition history explicit. The uncertainty-channel ablation suggests that the fitter's own σ estimate is not a reliable enough summary. A better engineered state is the executed-TI coverage histogram: it tells the policy which timing regions have already been sampled, while remaining compact and order-invariant. A recurrent policy is the learned alternative, but it is harder to debug and should be compared against the histogram baseline rather than used alone.

The most important scientific extension is to widen the action space and repeat the experiment beyond the T1 plate. First, the same adaptive framework should be tested with different sequence families: inversion-recovery spin echo, ordinary spin echo, multi-spin-echo trains, and possibly spoiled-GRE-like readouts. In parallel, the target should expand from the T1 spheres to the T2 and PD plates, where the informative acquisition parameters are different. The next step is a combined multi-parameter task in which the policy chooses not only TI and TR, but also echo time, echo-train structure, flip angle, and possibly readout budget. That would test a more interesting adaptive question: whether the next block should reduce T_1 uncertainty, T_2 uncertainty, PD uncertainty, or spatial uncertainty. This is closer to magnetic resonance fingerprinting, where sequence schedules are designed to make tissue trajectories distinguishable [12], [13]. It also requires a more general fitter or dictionary model, because the current fitter assumes one generalized IR spin-echo signal family.

Finally, the objective could become a Pareto curve rather than one fixed scan time. In clinical use, a protocol that reaches 5% MAPE in 120 s may be more valuable than one that reaches 3.5% in 240 s. A stop action or explicit scan-time penalty would let the policy choose this trade-off, while fixed baselines can be compared across the same accuracy–time frontier.

5.10 Summary

This chapter produced two conclusions. First, multi-fidelity training is a necessary engineering mechanism for simulation-in-the-loop RL with KomaMRI. It made the experiments feasible and exposed where simulator bias affects policy selection. Second, adaptivity only showed a clear quantitative advantage on the task designed to require it. The full 14-sphere agent learned a state-dependent policy but lost to fixed schedules. The five-sphere continuous-T_1 agent beat the fixed log-grid comparator at 240 s, using a compact probe-then-refine strategy. That is the central positive result, and the limitations above define the work needed to turn it from a controlled demonstration into a robust qMRI sequence-design method.

A

Technical Derivation Notes

This appendix records technical derivations that are useful for understanding the library implementation, but too detailed for the main digital-twin chapter. The first sections derive the fitting models from the Bloch-equation free-precession solutions under two assumptions: RF pulses are treated as instantaneous rotations, and residual transverse magnetisation is spoiled before the next block. Under those assumptions the fitters only need to carry the scalar longitudinal magnetisation M_z between shots.

A.1 Spin-Echo T2 Fit

For a spin echo, the 90° pulse tips equilibrium magnetisation into the transverse plane. The 180° pulse refocuses reversible static dephasing, so the echo magnitude is governed by irreversible T2 decay:

$$|S(\mathrm{TE})| = S_0 e^{-\mathrm{TE}/T_2}. \tag{A.1}$$

Taking logs gives a straight-line model,

$$\log|S| = \log S_0 - \frac{\mathrm{TE}}{T_2}, \tag{A.2}$$

which is the complete model used by `fit_t2_se`.

A.2 Inversion-Recovery T1 Fit

For long-TR inversion recovery, a preparation pulse of angle θ_{inv} leaves $M_z = M_0 \cos\theta_{\mathrm{inv}}$. Recovery for inversion time TI therefore gives

$$M_z(\mathrm{TI}) = M_0 \left[1 - (1 - \cos\theta_{\mathrm{inv}})\, e^{-\mathrm{TI}/T_1}\right]. \tag{A.3}$$

For a textbook 180° inversion, this reduces to $M_0(1 - 2e^{-\mathrm{TI}/T_1})$. Magnitude reconstruction removes the sign around the null point, so the practical conventional estimator fits

$$|S| = \left|A - Be^{-\mathrm{TI}/T_1}\right|. \tag{A.4}$$

This is the model used by `fit_t1_ir`. It is intentionally simple and is only appropriate for the conventional long-TR baseline.

A.3 Finite-TR and Transient IR Models

The RL acquisitions cannot assume long TR or fixed flip angles. For one IR-prepared block, define

$$E_1 = e^{-\mathrm{TI}/T_1}, \qquad E_2 = e^{-(\mathrm{TR}-\mathrm{TI})/T_1}. \tag{A.5}$$

With inversion angle θ_{inv} and excitation angle α_{exc}, the longitudinal recurrence is

$$M_z^{\mathrm{TI}} = (1 - E_1) + \cos(\theta_{\mathrm{inv}}) E_1 M_z^{\mathrm{pre}}, \tag{A.6}$$

$$M_z^{\mathrm{pre}\prime} = (1 - E_2) + \cos(\alpha_{\mathrm{exc}}) E_2 M_z^{\mathrm{TI}}. \tag{A.7}$$

Solving the fixed point gives `steady_state_mz_at_excite`. Iterating the recurrence for the first `Npe` shots and averaging M_z^{TI} gives `transient_mz_at_excite_npe`. This second model matches the actual 2D IR-SE acquisition used in E2: one phase-encode row is acquired per shot, starting from thermal equilibrium.

The measured transverse signal also contains the excitation projection $\sin(\alpha_{\mathrm{exc}})$. When excitation

angles vary across blocks, the caller divides magnitudes by this known factor before fitting; otherwise the fitter would confuse flip-angle scaling with T1 contrast.

A.4 Joint T1/T2 Identifiability

Joint T1/T2 fitting becomes possible only when TE varies. The IR-TSE model is

$$|S| = \left| A\, M_z^{\mathrm{exc}}(T_1; \mathrm{TI}, \mathrm{TR}, \theta_{\mathrm{inv}}, \alpha_{\mathrm{exc}}) e^{-\mathrm{TE}/T_2} \right|. \tag{A.8}$$

If TE is constant, the exponential T2 factor is absorbed into the fitted amplitude A, so T2 is unidentifiable. This is why `fit_t1_t2_generalized_ir` is reserved for multi-echo schedules and reports a wide T2 uncertainty for constant-TE data.

A.5 Finite K-space Sampling and Truncation Artefacts

A.5.1 Why Reconstruction Blurs and Rings

Chapter 2 established that an MR image is recovered by applying an inverse Fourier transform to sampled k-space. That Fourier relationship is exact only for infinite k-space. In practice, we acquire a finite $N_{pe} \times N_{fe}$ window, so the reconstructed image is not the true spin density $\rho(\mathbf{r})$ but a degraded version of it. The cleanest way to quantify the degradation is to model finite sampling as the full, infinite k-space multiplied by a rectangular box function: equal to 1 inside the sampled window and 0 outside.

The consequence follows from the convolution theorem: multiplication in one domain is convolution in the other. For a Fourier pair, image space $\leftrightarrow$ k-space,

$$\mathcal{F}\{f \cdot g\} = \mathcal{F}\{f\} * \mathcal{F}\{g\}. \tag{A.9}$$

So multiplying the true k-space by a box is equivalent, in the image domain, to convolving the true image with the Fourier transform of that box. That transform is the system's point-spread function (PSF): each ideal point in the object is smeared into a small PSF-shaped blob, and where the object has a sharp edge those blobs fail to add up cleanly, producing oscillations: Gibbs ringing.

An intuitive way to read the convolution theorem is that a convolution is "slide, multiply, integrate". Under a Fourier transform each shift becomes a phase factor e^{-ikx}, the integral collapses the slide, and only a pointwise product survives. This is also why FFT-based convolution is fast: transform, multiply, transform back.

For a rectangular window of N samples the PSF is the Dirichlet kernel, the periodic sinc:

$$\mathrm{PSF}(x) = \frac{\sin(\pi N x/\mathrm{FOV})}{N \sin(\pi x/\mathrm{FOV})}. \tag{A.10}$$

Reading it as numerator over denominator explains its shape. Let the pixel spacing be $\Delta = \mathrm{FOV}/N$.

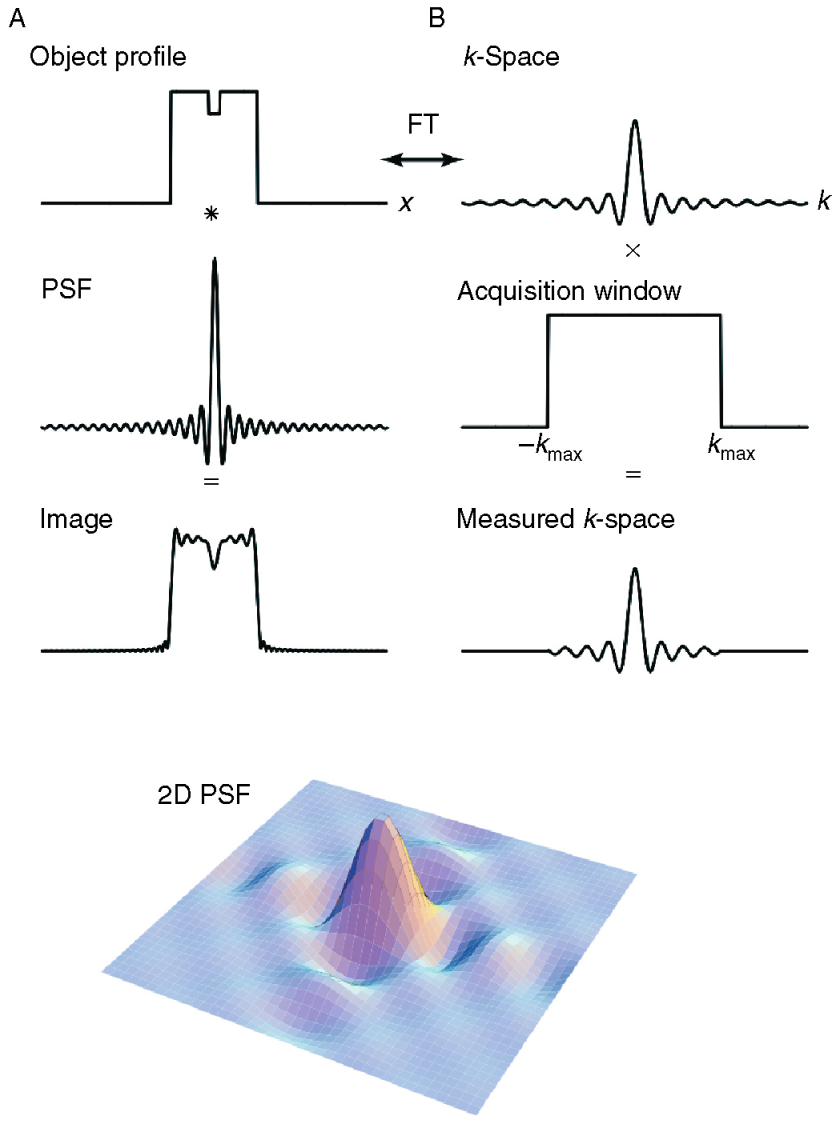


Figure A.1: The point-spread function. Producing a blurred image can be viewed equivalently in the image domain or in k-space. The limited extent of sampling in k-space is described by multiplying the full k-space distribution by a windowing function that removes high spatial frequencies. The resulting image is the convolution of the true image with the Fourier transform of that windowing function, the PSF. Figure adapted from [8].

Quantity	Scaling with N
Main-lobe width, physical units	FOV$/N$, shrinks as $1/N$
Main-lobe width, pixels	1 pixel, always
First side-lobe height	$\approx -21\%$, always
Side-lobe decay	$1/x$, always
Number of side-lobes across the FOV	grows as N

Table A.1: Increasing the sampled matrix size improves physical resolution, but the rectangular-window PSF has the same side-lobe structure in pixel units.

- **Unit peak at $x = 0$.** Both numerator and denominator vanish. L'Hopital's rule gives the ratio $\to 1$, so the centre pixel passes through at full weight. The kernel is normalised; no rescaling of ρ is required.
- **Zeros one pixel apart.** The numerator vanishes when $\pi N x/\mathrm{FOV} = m\pi$, i.e. at $x = m\,\mathrm{FOV}/N = m\Delta$. The main lobe is always exactly one pixel wide.
- **Fixed side-lobes.** The first negative side-lobe is approximately -21% of the peak, or about -13 dB, with a $1/x$ decay. This is set purely by the rectangular window shape.

A useful consequence is that, because the zeros sit on integer pixel offsets, an on-grid point source leaks nothing into its neighbours at the sample points. Ringing only becomes visible at a discontinuity, where the contributions on either side of the edge no longer cancel and the side-lobes ring across neighbouring pixels. This is the classic $\approx 9\%$ Gibbs overshoot on each side of a sharp edge.

A.5.2 Increasing the Matrix Does Not Fix It

The natural reflex is to sample more of k-space, i.e. to use a bigger box. However, a larger N only rescales the ringing; it does not remove it.

In pixel units the PSF shape barely changes: a larger N packs the same ringing into a finer physical scale, giving better resolution, and as $N \to \infty$ the main lobe collapses towards a delta function, recovering the ideal image. But the -21% side-lobe is intrinsic to the rectangular window. To actually suppress ringing, one must change the window shape, not its size. This is apodisation.

A.5.3 Apodisation: The Hamming Window

If a hard rectangular cut is what produces the strong side-lobes, then softening the edge of the k-space window should reduce them. This is apodisation: weighting k-space by a function that tapers smoothly to zero at the edges rather than truncating abruptly. Many window functions exist; a standard choice is the Hamming window,

$$w(n) = 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right), \qquad n = 0, \ldots, N-1. \tag{A.11}$$

which retains full weight near the centre, DC, and tapers to 0.08 at the k-space edges. Because the high spatial frequencies live at those edges, down-weighting them trades resolution for smoothness: the Fourier transform of the Hamming window has a main lobe roughly twice as wide, giving a blurrier image, but side-lobes suppressed from -13 dB to -43 dB, giving far less ringing.

The effect on an actual edge is the case that matters for the phantom, whose water–sphere boundaries are the sharpest features in the scene. The box PSF's side-lobes fail to cancel across the step and produce the $\approx 9\%$ Gibbs overshoot; the Hamming PSF's far weaker side-lobes suppress it, at the price of a softer, wider edge transition.

Two practical remarks close the loop. First, apodisation is a reconstruction-side operation: it is an element-wise multiply applied to the acquired k-space before the inverse FFT, costing a fraction of a

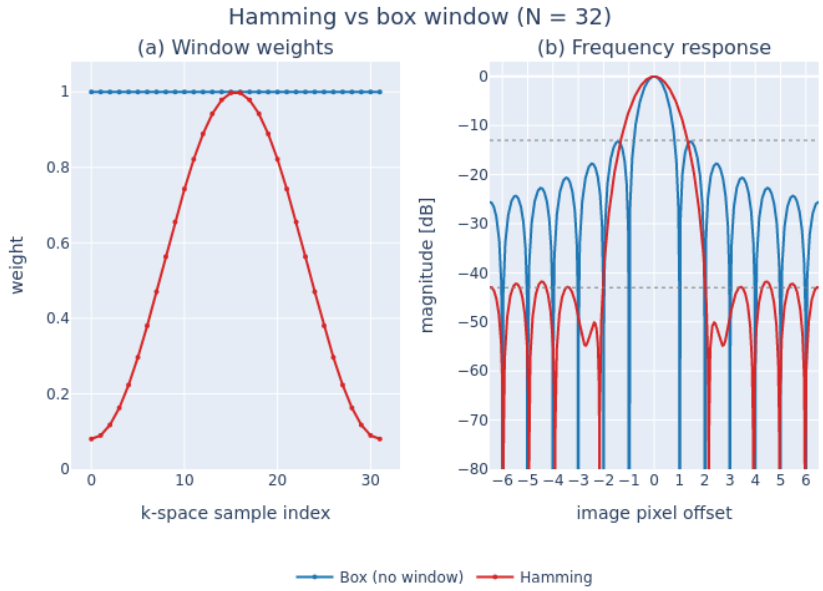


Figure A.2: **Hamming apodisation: window weights and their point-spread response. (a)** K-space weighting applied before reconstruction for $N = 32$ phase-encode lines. The box window, i.e. no apodisation, weights every line equally at 1.0; the Hamming window $0.54 - 0.46\cos(2\pi n/(N-1))$ tapers smoothly to 0.08 at the k-space edges, retaining full weight only near the centre. **(b)** Corresponding point-spread functions, shown as magnitude in decibels normalised to the main-lobe peak. The box window's abrupt truncation produces -13 dB side-lobes that ring across neighbouring pixels; the Hamming taper suppresses these to -43 dB at the cost of an approximately $2\times$ wider main lobe, i.e. reduced spatial resolution.

millisecond and entirely independent of the expensive Bloch simulation. It can therefore be toggled freely without re-acquiring or re-simulating. Second, it must not be confused with zero-padding. Padding k-space with zeros before the FFT interpolates the image onto a finer grid but adds no new measured frequencies, so it does not remove ringing. It only resamples the same Dirichlet PSF more densely. Genuine suppression requires changing the window shape, which is exactly what the phantom library's Hamming option does, and the mechanism here is what underpins its use against the coarse-water truncation artefact in Chapter 3.

A.6 E2 Noise Calibration

The E2 noise model adds independent complex Gaussian noise of absolute standard deviation σ to every k-space sample. Because the noise-free signal scale is a linear sum over simulated spins, the mapping from σ to image SNR depends on the spin count, and therefore on voxel size, slice thickness, matrix size and whether background water is included. A σ calibrated at one geometry does not transfer to another: for example, slicing a 1 mm slab through the T1 plate keeps 4,816 of the ∼27k full-3D sphere spins, shrinking the k-space signal scale by a factor of ∼5.6. The operating σ was therefore derived by a sweep at exactly the training configuration (T15 field, 1 mm slab, 1 mm voxels, 64×32): for each σ, 25 noise seeds were simulated under the Cramer–Rao-optimal fixed schedule and the resulting T_1-fit MAPE and three SNR conventions were recorded.

Of the SNR conventions, the NEMA MS-1 dual-acquisition measurement (signal from the mean of two acquisitions, noise from their difference image) is the one comparable with clinical phantom QA; the single-image NEMA variant reads systematically low on coarse matrices because Gibbs ringing inflates the ROI variance. Figure A.4 summarises the sweep. At $\sigma = 50$ the dual-acquisition SNR is ≈27 on the brightest spheres (the peak convention plotted on the figure's top axis) and ≈17 averaged across the T1 plate at the reference block (`scripts/e2_snr_report.jl`). Both bracket the in-vivo brain IR range

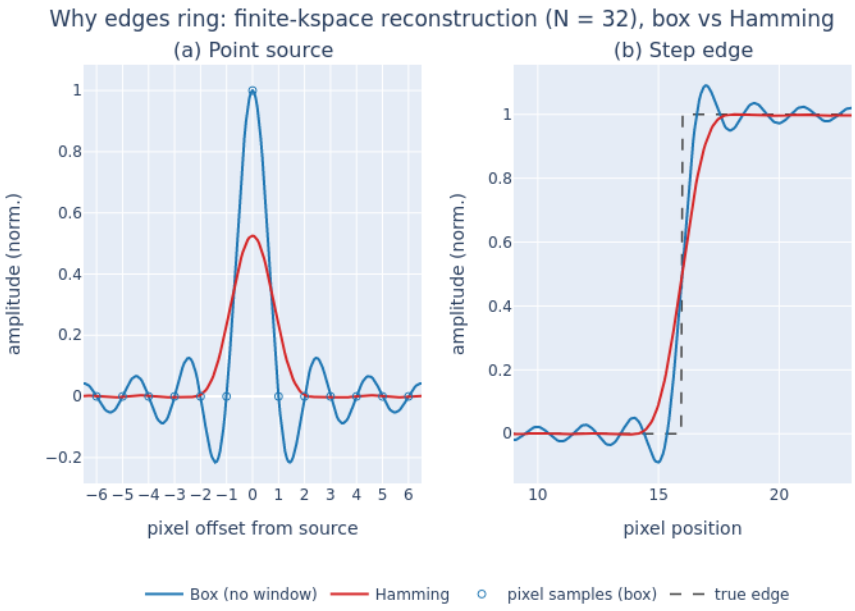


Figure A.3: **Why finite k-space sampling rings, and how Hamming apodisation suppresses it.** One-dimensional reconstruction from a finite acquisition of $N = 32$ phase-encode lines, comparing no window, or box truncation, with a Hamming window. **(a)** Point-source response: the reconstruction PSF. The box truncation yields a Dirichlet, sinc-like PSF whose zero-crossings fall exactly on integer pixel offsets. An on-grid point leaks nothing into neighbouring pixels at the sample points. The Hamming PSF has an approximately $2\times$ wider main lobe, nonzero at ± 1 px, so it blurs adjacent pixels, and a lower peak reflecting the window's DC attenuation. **(b)** The same two PSFs convolved with a step edge. The box PSF's -13 dB side-lobes fail to cancel across the discontinuity, producing Gibbs ringing with $\approx 9\%$ overshoot on both sides. The Hamming window's -43 dB side-lobes suppress this ringing, at the cost of a softer, wider edge transition. Equivalently, panel (a) is the PSF and panel (b) is that PSF convolved with an edge: the point-spread function explains the edge artefact.

of roughly 16–30 at 1.5 T, and the fixed-schedule fit error at $\sigma = 50$ is still close to its noise-free floor. Fit quality degrades gracefully up to $\sigma \approx 85$ and collapses beyond, so $\sigma = 50$ stresses the agent with clinically realistic noise without making the task estimation-limited.

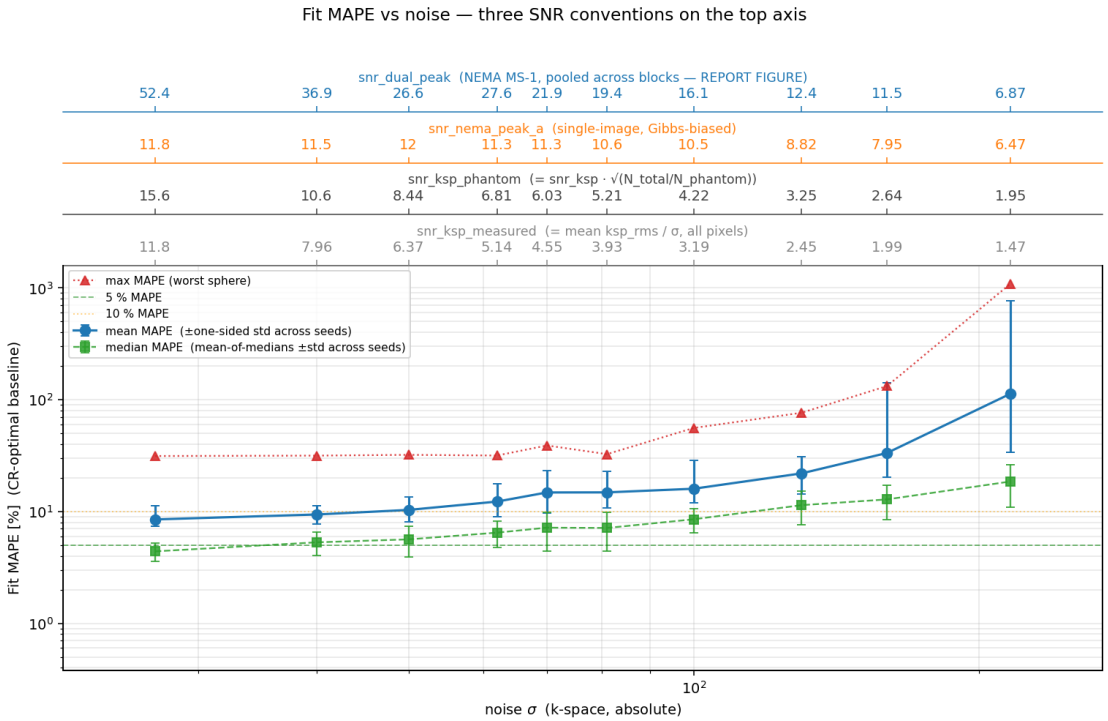


Figure A.4: **Noise calibration sweep at the E2 training geometry** (25 seeds per σ, Cramer–Rao-optimal fixed schedule). Mean, median and worst-sphere T_1-fit MAPE versus absolute k-space σ, with the corresponding SNR conventions on the top axes. The NEMA MS-1 dual-acquisition row (top, blue) is the clinically comparable measurement; the operating point $\sigma = 50$ corresponds to peak dual-acquisition SNR ≈27. The worst-sphere curve is dominated by the longest-T_1 sphere, which the fixed schedule cannot constrain even at $\sigma = 0$.

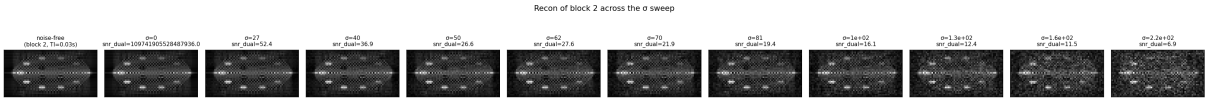


Figure A.5: **Visual artefact level across the noise sweep.** Reconstructed magnitude images of one acquisition block (TI = 0.03 s) as σ increases left to right, annotated with the measured dual-acquisition SNR. At the operating point $\sigma = 50$ (peak SNR ≈27) the spheres remain clearly resolved over a visibly noisy background; by $\sigma \geq 160$ the background noise approaches the sphere contrast and fits collapse.

A.7 Cached Background-Water Validation

This appendix supports Section 5.4.2: it shows why the background water must be Bloch-accurate but does not need to be re-simulated every step, and quantifies the accuracy of the cached-template model used by the `cached` fidelity rungs.

Figure A.6 compares reconstructions of the same acquisition under four water models. Two cached variants are shown: the *scalar* variant rescales the whole template by a single shot amplitude, while the *per-line* variant rescales each k-line by its own shot amplitude; E2 uses the per-line variant, which is the exact factorisation for a multi-shot acquisition. Both cached variants are visually indistinguishable from the full simulation (their signed difference images are blank at this colour scale), whereas analytic water leaves structured residuals across both the spheres and the background. Figure A.7 quantifies the consequence for the quantity that actually matters, the fitted T_1: the cached models stay at or below the

~1% fitter grid floor on every sphere, while analytic water produces 4–40% per-sphere deviations and even the fully analytic forward model 2–16%. An analytic-water rung would therefore corrupt the reward signal; the cached rung does not.

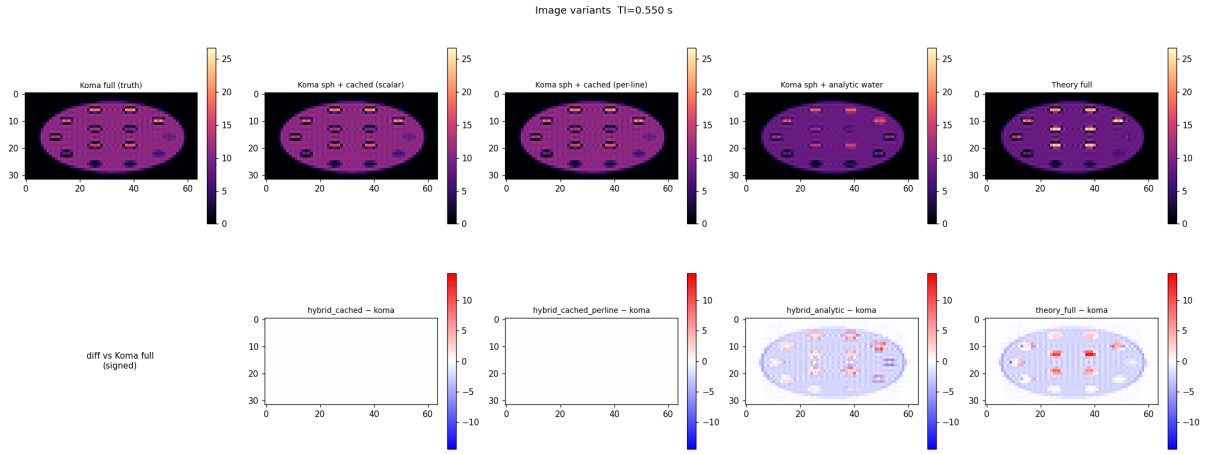


Figure A.6: **Water-model comparison at matched sequence settings** (TI = 0.55 s, $\alpha = 30°$, 64×32). Top row: reconstructed magnitude images for the full Bloch simulation (truth), Bloch spheres with cached water (scalar and per-line variants), Bloch spheres with analytic water, and the fully analytic forward model. Bottom row: signed difference against the full simulation. The cached variants are exact at this colour scale; the analytic water model leaves structured sphere and background residuals that no TI rescaling can remove.

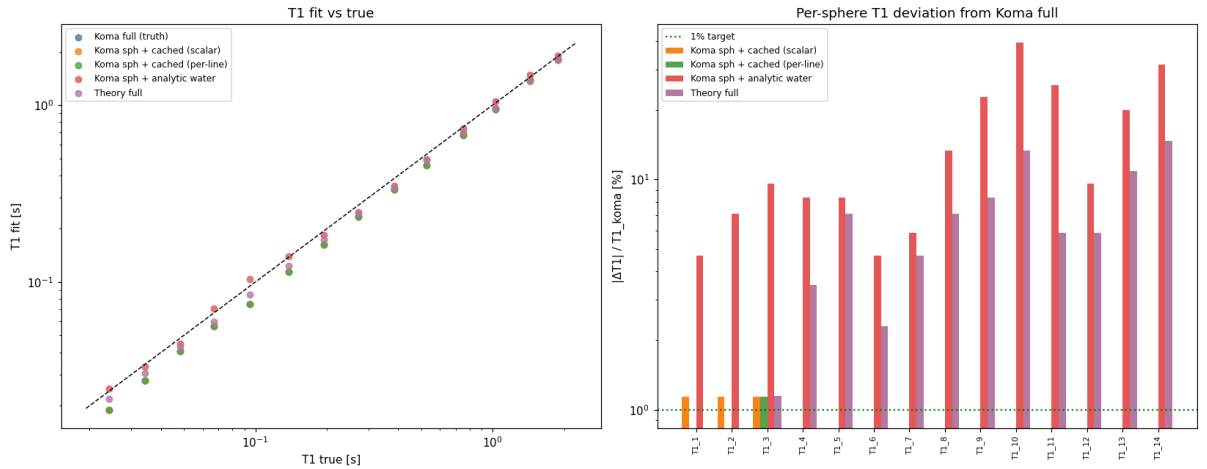


Figure A.7: **Per-sphere T_1-fit impact of the water model.** Left: fitted versus true T_1 for each variant. Right: per-sphere deviation from the full-Bloch fit. The cached variants sit at or below the ~1% fitter grid floor on every sphere (most bars are below the axis); analytic water deviates by 4–40%, worst at short T_1, and the fully analytic model by 2–16%.

Figure A.8 validates the α-template bank of Section 5.4.2 (36 templates spanning the action range) across TI and flip angle. The cached water k-space matches the Bloch water to 10^{-3}–10^{-2} relative error over the operating TI range, with two readable features: the error collapses to $\sim 10^{-8}$ at the template's reference TI (where the factorisation is evaluated at its construction point), and it rises towards TI $\approx$ 1.6–2 s, where the water signal approaches its own inversion null and the relative-error denominator vanishes. For context, the right panel shows that modelling the water without B_0 inhomogeneity changes the water k-space by 10–40%: the cache approximation error is one to two orders of magnitude below a routine modelling choice, so it is not the binding fidelity constraint. End-to-end, cached-water T_1 fits match the full simulation to 0.12% on average (worst sphere 1.15%, the fitter's discrete-grid floor), and the per-step speedup is 8.0× (4.47 s → 0.56 s at the benchmark settings) for a one-off bank build cost of 153 s.

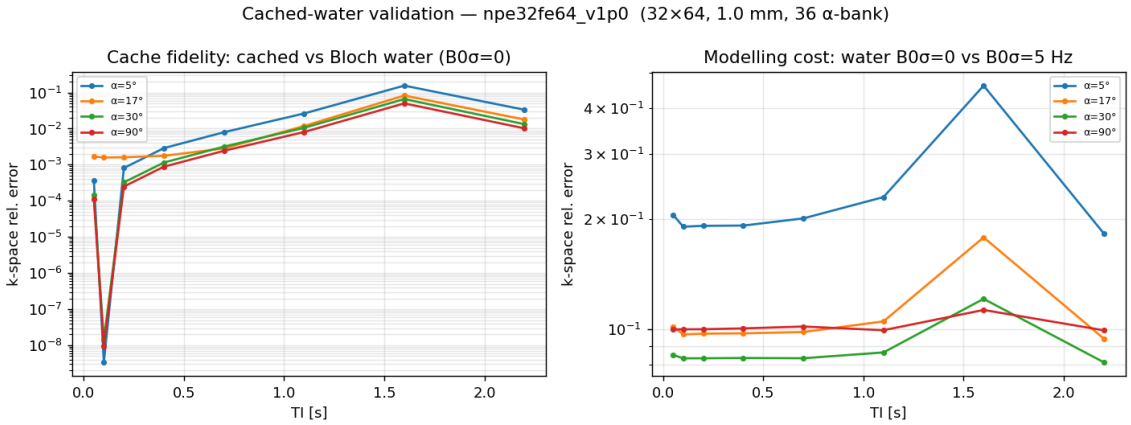


Figure A.8: α**-bank cache fidelity across TI and flip angle** (36-template bank, 64×32, 1 mm). Left: relative k-space error of cached versus Bloch water at $\alpha \in \{5°, 17°, 30°, 90°\}$; the dip is the template's reference TI and the rise towards TI $\approx$ 1.6–2 s reflects the water signal approaching its inversion null. Right: the error made by modelling water without B_0 inhomogeneity ($B_0\sigma = 0$ versus 5 Hz), one to two orders of magnitude larger than the cache error at matched settings.

Bibliography

[1] W. C. R. Fund, *UK cancer statistics*, en-GB. [Online]. Available: https://www.wcrf.org/preventing-cancer/cancer-statistics/uk-cancer-statistics/ (visited on 01/21/2026).

[2] R. Atun, D. A. Jaffray, and M. B. e. a. Barton, "Expanding global access to radiotherapy," *The Lancet Oncology*, vol. 16, no. 10, pp. 1153–1186, Sep. 2015. DOI: 10.1016/s1470-2045(15)00222-3. [Online]. Available: https://doi.org/10.1016/s1470-2045(15)00222-3.

[3] X. Liu, Z. Li, and Y. Yin, "Clinical application of MR-Linac in tumor radiotherapy: A systematic review," *Radiation Oncology*, vol. 18, no. 1, p. 52, Mar. 2023, ISSN: 1748-717X. DOI: 10.1186/s13014-023-02221-8. [Online]. Available: https://doi.org/10.1186/s13014-023-02221-8.

[4] J. de Leon, T. Twentyman, M. Carr, M. Jameson, and V. Batumalai, "Optimising the mr-linac as a standard treatment modality," *Journal of Medical Radiation Sciences*, vol. 70, no. 4, pp. 491–497, 2023. DOI: https://doi.org/10.1002/jmrs.712. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/jmrs.712. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/jmrs.712.

[5] R. Heckel, M. Jacob, A. Chaudhari, O. Perlman, and E. Shimron, "Deep learning for accelerated and robust MRI reconstruction," *Magnetic Resonance Materials in Physics, Biology and Medicine*, vol. 37, no. 3, pp. 335–368, Jul. 2024, ISSN: 1352-8661. DOI: 10.1007/s10334-024-01173-8. [Online]. Available: https://doi.org/10.1007/s10334-024-01173-8.

[6] D. B. Plewes and W. Kucharczyk, "Physics of mri: A primer," *Journal of Magnetic Resonance Imaging*, vol. 35, no. 5, pp. 1038–1054, 2012. DOI: https://doi.org/10.1002/jmri.23642. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/jmri.23642. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/jmri.23642.

[7] R.-J. M. van Geuns, P. A. Wielopolski, H. G. de Bruin, *et al.*, "Basic principles of magnetic resonance imaging," *Progress in Cardiovascular Diseases*, vol. 42, no. 2, pp. 149–156, 1999. DOI: https://doi.org/10.1016/S0033-0620(99)70014-9. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0033062099700149.

[8] R. B. Buxton, "Mapping the mr signal," in *Introduction to Functional Magnetic Resonance Imaging: Principles and Techniques*. Cambridge University Press, 2009, pp. 205–231.

[9] K. J. Layton, S. Kroboth, F. Jia, *et al.*, "Pulseq: A rapid and hardware-independent pulse sequence prototyping framework," *Magnetic Resonance in Medicine*, vol. 77, no. 4, pp. 1544–1552, 2017. DOI: https://doi.org/10.1002/mrm.26235. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/mrm.26235. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/mrm.26235.

[10] C. Castillo-Passi, R. Coronado, G. Varela-Mattatall, C. Alberola-López, R. Botnar, and P. Irarrazaval, "Komamri.jl: An open-source framework for general mri simulations with gpu acceleration," *Magnetic Resonance in Medicine*, vol. 90, no. 1, pp. 329–342, 2023. DOI: https://doi.org/10.1002/mrm.29635. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/mrm.29635. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/mrm.29635.

[11] A. Loktyushin, K. Herz, N. Dang, *et al.*, "Mrzero - automated discovery of mri sequences using supervised learning," *Magnetic Resonance in Medicine*, vol. 86, no. 2, pp. 709–724, 2021. DOI: https://doi.org/10.1002/mrm.28727. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/mrm.28727. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/mrm.28727.

[12] B. Mehta, S. Coppo, D. F. McGivney, *et al.*, "Magnetic resonance fingerprinting: A technical review," *Magnetic Resonance in Medicine*, vol. 81, no. 1, pp. 25–46, 2019. DOI: 10.1002/mrm.27403.

[13] S. P. Jordan, S. Hu, I. Rozada, *et al.*, "Automated design of pulse sequences for magnetic resonance fingerprinting using physics-inspired optimization," *Proceedings of the National Academy of Sciences*, vol. 118, no. 40, e2020516118, 2021. DOI: 10.1073/pnas.2020516118. eprint: https://www.pnas.org/content/118/40/e2020516118.full.pdf. [Online]. Available: https://www.pnas.org/content/118/40/e2020516118.

[14] A. Ocanto, L. Torres, M. Montijano, *et al.*, "MR-LINAC, a New Partner in Radiation Oncology: Current Landscape," *Cancers*, vol. 16, no. 2, p. 270, Jan. 2024, ISSN: 2072-6694. DOI: 10.3390/cancers16020270. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10813892/ (visited on 01/20/2026).

[15] A. Kuisma, I. Ranta, J. Keyriläinen, *et al.*, "Validation of automated magnetic resonance image segmentation for radiation therapy planning in prostate cancer," *Physics and Imaging in Radiation Oncology*, vol. 13, pp. 14–20, 2020, ISSN: 2405-6316. DOI: https://doi.org/10.1016/j.phro.2020.02.004. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S240563162030004X.

[16] A. D. Elster, *Magnetism*, en. [Online]. Available: http://mriquestions.com/ai-in-clinical-mri.html (visited on 01/20/2026).

[17] B. Zhu, J. Liu, N. Koonjoo, B. R. Rosen, and M. S. Rosen, "Automated pulse sequence generation (autoseq) using bayesian reinforcement learning in an mri physics simulation environment," in *Proceedings of the Joint Annual Meeting ISMRM-ESMRMB*, 2018, p. 0438.

[18] I. Beracha, A. Seginer, and A. Tal, "Adaptive model-based magnetic resonance," *Magnetic Resonance in Medicine*, vol. 90, no. 3, pp. 839–851, 2023. DOI: https://doi.org/10.1002/mrm.29688. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/mrm.29688. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/mrm.29688.

[19] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html.

[20] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Advances in Neural Information Processing Systems*, vol. 12, 1999. [Online]. Available: https://papers.nips.cc/paper/1713-policy-gradient-methods-for-reinforcement-learning-with-function-approximation.

[21] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, pp. 229–256, 1992. DOI: 10.1007/BF00992696. [Online]. Available: https://doi.org/10.1007/BF00992696.

[22] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *International Conference on Learning Representations*, 2016. DOI: 10.48550/arXiv.1506.02438. arXiv: 1506.02438. [Online]. Available: https://arxiv.org/abs/1506.02438.

[23] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proceedings of the 32nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 37, PMLR, 2015, pp. 1889–1897. [Online]. Available: https://proceedings.mlr.press/v37/schulman15.html.

[24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347. [Online]. Available: https://arxiv.org/abs/1707.06347.

[25] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. DOI: 10.1162/neco.1997.9.8.1735.

[26] S. Walker-Samuel, "Using deep reinforcement learning to actively, adaptively and autonomously control a simulated mri scanner," in *Proceedings of the Joint Annual Meeting ISMRM-ESMRMB*, 2019, p. 0473. [Online]. Available: https://cds.ismrm.org/protected/19MProceedings/PDFfiles/0473.html.

[27] M. Besançon, T. Papamarkou, D. Anthoff, *et al.*, "Distributions.jl: Definition and modeling of probability distributions in the juliastats ecosystem," *Journal of Statistical Software*, vol. 98, no. 16, pp. 1–30, 2021. DOI: 10.18637/jss.v098.i16. [Online]. Available: https://www.jstatsoft.org/article/view/v098i16.

[28] C.-H. Sohn, R. J. Sevick, R. Frayne, H.-W. Chang, S.-P. Kim, and D.-K. Kim, "Fluid attenuated inversion recovery (flair) imaging of the normal brain: Comparisons between under the conditions of 3.0 tesla and 1.5 tesla," *Korean Journal of Radiology*, vol. 11, no. 1, pp. 19–24, 2010. DOI: 10.3348/kjr.2010.11.1.19. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC2799645/.

[29] K. Murphy, J. Bodurka, and P. A. Bandettini, "How long to scan? the relationship between fmri temporal signal to noise ratio and necessary scan duration," *NeuroImage*, vol. 34, no. 2, pp. 565–574, 2007. DOI: 10.1016/j.neuroimage.2006.09.032. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC2223273/.

[30] H. Sifaou and O. Simeone, *Multi-fidelity hybrid reinforcement learning via information gain maximization*, 2025. arXiv: 2509.14848 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2509.14848.

[31] M. Cutler, T. J. Walsh, and J. P. How, "Real-world reinforcement learning via multifidelity simulators," *IEEE Transactions on Robotics*, vol. 31, no. 3, pp. 655–671, 2015. DOI: 10.1109/TRO.2015.2419431.

[32] S. Khairy and P. Balaprakash, "Multi-fidelity reinforcement learning with control variates," *Neurocomputing*, vol. 597, p. 127 963, 2024. DOI: 10.1016/j.neucom.2024.127963.

[33] V. Suryan, N. Gondhalekar, and P. Tokekar, "Multi-fidelity reinforcement learning with gaussian processes: Model-based and model-free algorithms," *IEEE Robotics & Automation Magazine*, vol. 27, no. 2, pp. 117–128, 2020. DOI: 10.1109/MRA.2020.2988800.